



**COMSATS University Islamabad,
Sahiwal Campus, Pakistan**

AI-Powered Career Assistant Platform

**A project presented to
COMSATS Institute Islamabad, Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2022-2026)

By

Syed Rizwan Ali Naqvi

CIIT/FA22-BSE-045/SWL

Wanya Kamran Qureshi

CIIT/FA22-BSE-047/SWL

Ishna Zakaullah

CIIT/FA22-BSE-058/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied from any source. It is further declared that we have developed this software and the accompanying report entirely based on our personal efforts. If any part of this project is proven to be copied from any source or found to be a reproduction of some other work. We will stand by the consequences. No Portion of the work presented has been submitted to any application for any other degree or qualification of this or any other university or institute of learning.

Syed Rizwan Ali

Wanya Kamran

Ishna Zaka

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(SE) “**AI-Powered Career Assistant Platform**” was developed by **Syed Rizwan Ali Naqvi (CIIT/FA22-BSE-045)**, **Wanya Kamran Qureshi(CIIT/FA22-BSE-047)** and **Ishna Zakaullah (CIIT/FA22-BSE-058)** under the supervision of “**Mam Hafsa Saleem**” and co supervisor “**Dr. Muhammad Shoaib**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

Dr. Hafsa Saleem

Lecturer

Department of Computer Science,
COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Muhammad Munawar Iqbal

Assistance Professor

Department of Computer Science,
UET Taxila

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science,
COMSATS University Islamabad, Sahiwal Campus

Executive Summary

The increasing competitiveness of modern job markets has created significant challenges for students, fresh graduates, and early-career professionals who must simultaneously manage job searching, CV preparation, skill development, and interview preparation without access to centralized, intelligent support tools. Existing platforms such as LinkedIn, Indeed, and Rozee.pk provide job listings but offer no automation, no personalized CV feedback, no structured interview preparation, and no integrated skill development guidance, leaving users to navigate these demands manually across multiple disconnected platforms.

To address these limitations, the **AI-Powered Career Assistant Platform** has been developed. It is a comprehensive, web-based intelligent system that consolidates ten career support modules into a single unified ecosystem accessible through any standard web browser. The platform automates job discovery by scraping listings from LinkedIn, Indeed, and Rozee.pk, analyzes uploaded CVs using natural language processing to extract skills, generate quality scores, and identify skill gaps, and automatically submits job applications on behalf of users whose profiles meet a defined match threshold.

The AI-Powered Career Assistant Platform further provides users with a conversational AI chatbot for real-time career guidance and job recommendations, a curated Resource Hub linking directly to Coursera and Udemy courses based on detected skill gaps, and a gamified Community Forum that fosters peer collaboration through badges and leaderboard rankings. An AI Mock Interview Module simulates structured technical interviews in Database and Object-Oriented Programming domains, evaluating user responses in real time and generating detailed performance feedback reports upon session completion.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Mam Hafsa Saleem,” and our Co-Supervisor, “Dr. Muhammad Shoaib”. Without their personal supervision, advice, and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Syed Rizwan Ali

Wanya Kamran

Ishna Zaka

Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
UI	User Interface
UX	User Experience
EMR	Electronic Medical Records
EHR	Electronic Health Records
HIS	Health Information System
OTP	One-Time Password
DB	Database
SQL	Structured Query Language
NoSQL	Not Only SQL
SDLC	Software Development Life Cycle
MVC	Model-View-Controller
NFR	Non-Functional Requirements
FR	Functional Requirements
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token

Table of Contents

1. Introduction.....	15
1.1 Brief.....	16
1.1.1. Purpose of the Project.....	17
1.1.2. Scope of the System	17
1.1.3. Technologies Used.....	18
1.1.4. System Overview	18
1.1.5. Outcome of the Project.....	18
1.2. Relevance to Course Modules	19
1.2.1. Software Engineering	19
1.2.2. Database Systems	19
1.2.3. Web Engineering and Internet Programming.....	20
1.2.4. Artificial Intelligence and Machine Learning	20
1.2.5. Object-Oriented Programming	20
1.2.6. Data Structures and Algorithms	21
1.2.7. Human-Computer Interaction.....	21
1.2.8. Information Security	21
1.2.9. Distributed Computing and Computer Networks.....	22
1.2.10. Final Year Project I and II.....	22
1.3. Project Background.....	22
1.4. Literature Review	24
1.5. Analysis from Literature Review	27
1.5.1. Alignment with Digital Career Support Trends	27
1.5.2. Comparison with Existing NLP-Based CV Analysis Research.....	27
1.5.3. Relevance to Job Recommendation and Matching Systems	27
1.5.4. Integration of Automated Application Submission	28
1.5.5. Application of Adaptive Assessment Principles.....	28
1.5.6. Validation of Gamification in Community Engagement.....	28
1.5.7. Addressing Limitations of Existing Healthcare Technologies.....	29
1.6. Methodology and Software Lifecycle	29
1.6.1. Methodology.....	29

1.6.2. Agile Practices Applied.....	30
1.6.3. Software Development Life Cycle (SDLC) Model.....	30
1.6.4. Product Backlog Management	30
1.6.5. Incremental Delivery.....	31
1.6.6. Rationale behind Selected Methodology	31
1.6.7. Development Tools and Environment	31
1.6.8. Testing and Quality Assurance	32
1.6.9. Advantages of the Selected Methodology and SDLC	32
2. Problem Definition.....	34
2.1 Problem Statement	35
2.2 Deliverables and Development Requirements.....	35
2.2.1 Deliverables.....	35
2.2.2 Development Requirements	37
2.3 Current System	38
2.3.1 Manual and Fragmented Job Search Process	38
2.3.2 Limitations of Existing Job Portals	38
2.3.3 Absence of Intelligent CV Analysis Tools.....	39
2.3.4 Lack of Automated Job Application Submission.....	39
2.3.5 Lack of Integrated Interview Preparation.....	40
2.3.6 Disconnection Between Skill Development and Career Planning	40
2.3.7 Absence of Community and Peer Support Infrastructure.....	40
2.4 Summary.....	41
3. Requirement Analysis.....	43
3.1 Introduction to Requirement Analysis.....	43
3.2 System Overview	44
3.3 Actors in the System.....	44
3.3.1 User	44
3.3.2 Administrator	45
3.4 Use Case Diagrams.....	45
3.5 Detailed Use Cases	49
3.5.1 User Registration and Login	49
3.5.2 Manage User Profile.....	51

3.5.3 Upload and Analyze CV	52
3.5.4 Search and Filter Job Listings	53
3.5.5 Enable Auto-Apply	54
3.5.6 Use AI Chatbot.....	56
3.5.7 Browse Resource Hub.....	57
3.5.8 Create Forum Post and Reply.....	58
3.5.9 Take Mock Interview Session	59
3.5.10 View Performance Feedback Report.....	60
3.5.11 Receive Notifications and Email Alerts	61
3.5.12 Admin -Manage User Accounts	62
3.5.13 Admin - Moderate Forum Content.....	63
3.5.14 Admin Monitor Job Scraper	64
3.5.15 Admin View Analytics and Export Audit Logs	65
3.6 Functional Requirements	66
3.6.1 User Authentication and Profile Management	67
3.6.2 CV Analysis and Skill Gap Detection.....	67
3.6.3 Job Discovery and Match Scoring.....	68
3.6.4 Auto-Apply Automation	68
3.6.5 AI Chatbot and Career Guidance	68
3.6.6 Resource Hub and Learning Recommendations	69
3.6.7 Community Forum and Gamification	69
3.6.8 Mock Interview Simulation.....	69
3.6.9 Notification System.....	70
3.6.10 Admin Panel and Governance.....	70
3.7 Non-Functional Requirements.....	70
3.7.1 Performance	70
3.7.2 Scalability.....	71
3.7.3 Security	71
3.7.4 Reliability and Availability	72
3.7.5 Usability	72
3.7.6 Maintainability	72
3.7.7 Compliance.....	72

3.7.8 Backup and Recovery.....	73
3.7.9 Performance Monitoring and Logging.....	73
4. Design and Architecture	75
4.1 System Architecture	75
4.1.1 Architecture Style.....	75
4.1.2 Components.....	75
4.2 Data Representation.....	79
4.2.1 Database Design.....	79
4.2.2 Entity-Relationship Representation.....	80
4.2.3 Data Flow	82
4.2.4 Entity Descriptions and Constraints	88
4.3 Process Flow / Representation.....	88
4.3.1 Login and Authentication Process.....	88
4.3.2 CV Upload and Analysis Process.....	89
4.3.3 Job Scraping and Match Scoring Process.....	89
4.3.4 Mock Interview Session Process.....	90
4.4 Design Models.....	90
4.4.1 Class Model Description	90
4.4.2 Sequence Model Description.....	90
4.4.3 Activity Model Description.....	91
5. Implementation	93
5.1 Algorithms	93
5.1.1 Authentication Algorithm	93
5.1.2 Mock Interview Session Initiation Algorithm.....	94
5.1.3 CV Analysis and Feedback Generation Algorithm	94
5.1.4 Auto-Apply Execution Algorithm.....	95
5.2 External APIs.....	95
5.3 User Interface	97
6.1 Manual Testing.....	106
6.1.1 System Testing	107
6.1.2 Unit Testing.....	108
6.1.3 Functional Testing.....	109

6.1.4 Integration Testing	111
6.2 Automated Testing	112
7. Conclusion	115
7.1 Future Work.....	115
7.1.1 Mobile Application	115
7.1.2 Multi-Platform Job Portal Integration	115
7.1.3 Premium Subscription Models	116
7.1.4 Advanced AI Analytics and Career Insights	116
7.1.5 Integration with Professional Networking Platforms	116
7.1.6 Enhanced Security and Privacy Features	116
References.....	117

List of Figures

Figure 1:Use Case Diagram - Authentication & CV Analyzer	46
Figure 2:Use Case Diagram - Job Scraper & Auto-Apply	47
Figure 3: Use Case Diagram - AI Chatbot, Resource Hub & Notifications	47
Figure 4: Use Case Diagram - Community Forum & Mock Interview	48
Figure 5: Use Case Diagram - Admin Panel	49
Figure 6: System Architecture - Presentation & Application Layers	78
Figure 7: System Architecture - AI/ML, Task Queue, Data & External Layers	79
Figure 8: Class Diagram Core Classes	80
Figure 9: Class Diagram - AI/ML & Feature Classes	81
Figure 10: User Authentication & CV Analysis Flow (Sequence Diagrams)	83
Figure 11: Job Scraping & Auto-Apply Flow (Sequence Diagrams).....	84
Figure 12: AI Chatbot & Mock Interview Flow (Sequence Diagrams)	86
Figure 13: Activity Diagram - User Journey	87
Figure 14: Landing / Home Page.....	97
Figure 15: How It Works Section.....	98
Figure 16: Modules Overview	98
Figure 17: Create Account Screen.....	99
Figure 18: Sign In Screen	100
Figure 19: User Dashboard.....	100
Figure 20: CV Analyzer Screen	101
Figure 21: CV Analyzer result details Screen	102
Figure 22: CV Analyzer result Screen.....	102
Figure 23: Jobs / Opportunities Screen	103
Figure 24: Courses / Learning Resources Screen.....	104

List of Tables

Table 1: Comparison of Existing Career Platforms and AI-Powered Career Assistant Platform	24
Table 2: Structured description of the User Registration and Login use case	50
Table 3: Structured description of the Manage User Profile use case	51
Table 4: Structured description of the Upload and Analyze CV use case	52
Table 5: Structured description of the Search and Filter Job Listings use case	53
Table 6: Structured description of the Enable Auto-Apply use case	55
Table 7: Structured description of the Use AI Chatbot use case	56
Table 8: Structured description of the Create Forum Post and Reply use case	58
Table 9: Structured description of the Take Mock Interview Session use case	59
Table 10: Structured description of the Receive Notifications and Email Alerts use case	61
Table 11: Structured description of the Admin Manage User Accounts use case	62
Table 12: Structured description of the Admin Moderate Forum Content use case	63
Table 13: Structured description of the Admin Monitor Job Scraper use case	64
Table 14: Structured description of the Admin View Analytics and Export Audit Logs	66
Table 15: Details of External APIs/Services Used in the Job Pilot Platform	96
Table 16: Unit Testing Table	108
Table 17: Functional Testing Table	110
Table 18: Integration Testing Table	111
Table 19: Automated Testing Table	113

Chapter 1

INTRODUCTION

1. Introduction

The job market in Pakistan and globally has grown increasingly competitive, placing significant pressure on students, fresh graduates, and early-career professionals who must simultaneously manage job searching, CV preparation, skill development, and interview readiness without access to centralized, intelligent support tools. A large proportion of graduates entering the workforce lack the guidance, resources, and practical tools needed to present themselves effectively to employers, identify the skills demanded by their target industries, and prepare adequately for technical interview processes. In many universities and educational institutions across Pakistan, career support services remain limited or absent, leaving graduates to navigate the complexities of professional employment independently and without personalized assistance.

The rapid growth of online job platforms such as LinkedIn, Indeed, and Rozee.pk has created an overwhelming volume of job opportunities that candidates must manually sift through across multiple disconnected platforms. Each platform operates in isolation, offering job listings but providing no mechanism for intelligent CV analysis, automated application submission, personalized skill gap detection, or structured interview preparation. As a result, candidates frequently submit poorly optimized resumes that fail to meet the keyword and formatting standards expected by modern recruitment processes, reducing their visibility to recruiters regardless of their actual technical competence. The absence of tools that can objectively evaluate a candidate's CV, recommend targeted learning resources, and simulate real interview conditions creates a significant barrier between capable graduates and the employment opportunities available to them.

The AI-Powered Career Assistant Platform is developed as a comprehensive response to these challenges. It is a full-stack web-based platform that consolidates ten integrated career support modules into a single, intelligent, and accessible ecosystem designed specifically to serve students, fresh graduates, and early-career professionals. The platform leverages advances in natural language processing, transformer-based semantic language models, and distributed task automation to deliver end-to-end career support covering CV analysis, automated job discovery, intelligent match scoring, career mentorship through an AI chatbot, personalized learning resource recommendations, peer collaboration through a gamified community forum, automated job

application submission, and adaptive mock interview simulation.

The system is built using Django for the backend, with Django's built-in templating engine handling frontend views through server-side rendered HTML and CSS. SQLite serves as the primary database for data persistence. Natural language processing capabilities are delivered through spaCy for CV skill extraction, job match scoring, chatbot response generation, and interview answer evaluation, with lightweight sentence embeddings supporting semantic similarity computation for job matching. Web scraping functionality is implemented using BeautifulSoup and Selenium for dynamic job data collection. The backend development environment is managed using a Python virtual environment to isolate dependencies and ensure consistent package versioning across the development workflow.

The AI-Powered Career Assistant Platform aims to reduce the time, effort, and expertise required for job seekers in Pakistan to navigate competitive employment markets. By centralizing all career support functions within a single intelligent web platform, the system enables graduates to optimize their professional profiles, discover and apply for relevant opportunities automatically, develop targeted skills based on real industry demand, and build interview confidence through AI-driven practice sessions. The platform represents a practical, scalable, and accessible solution to the career readiness challenges faced by Pakistan's growing graduate population, and demonstrates the meaningful application of artificial intelligence, software engineering, and distributed systems within a unified career support ecosystem.

The AI-Powered Career Assistant Platform, therefore, represents more than a technical achievement. It is a meaningful step toward addressing the systemic career readiness gap faced by Pakistan's growing graduate population.

1.1 Brief

The AI-Powered Career Assistant Platform is a comprehensive full-stack web application developed as a Final Year Project for the Bachelor of Science in Software Engineering program at COMSATS University Islamabad, Sahiwal Campus. The platform is designed to address the fragmented and largely manual nature of job searching, CV optimization, skill development, and

interview preparation currently experienced by students and graduates in Pakistan. By integrating ten functional modules within a single intelligent system, the platform delivers an end-to-end career support solution powered by natural language processing, transformer-based AI models, and task automation frameworks.

1.1.1. Purpose of the Project

The primary purpose of the AI-Powered Career Assistant Platform is to eliminate the fragmentation experienced by job seekers who must independently manage CV preparation, job searching, skill development, and interview preparation across multiple disconnected and non-intelligent platforms. The platform centralizes all these functions and applies AI-driven intelligence to each, enabling users to receive objective CV feedback, discover relevant job opportunities automatically, identify and close skill gaps through targeted learning resources, obtain real-time career guidance through a conversational AI mentor, and build technical interview confidence through adaptive mock interview sessions, all within a single, freely accessible web interface.

1.1.2. Scope of the System

The scope of the AI-Powered Career Assistant Platform encompasses ten integrated modules accessible through a Django-powered web application. The system serves two primary user roles: the registered User, who accesses all career support features, and the Administrator, who governs platform operations through Django's built-in admin panel. All module interactions are handled through Django's server-side view layer, with HTML templates rendering responses directly to the browser.

The system integrates with three external job portals, LinkedIn, Indeed, and Rozee.pk for automated job data retrieval, and with Coursera and Udemy for learning resource recommendations. All asynchronous operations, including job scraping, CV analysis, auto-apply submission, and email digest generation, are managed through a Celery and Redis task queue infrastructure. The system does not include ATS integration, video-based interviews, native mobile applications, payment processing, or job placement guarantees within its current scope.

1.1.3. Technologies Used

The platform is developed using a straightforward, maintainable technology stack. Django 4.x forms the backend application layer, with Django's built-in templating engine delivering server-side rendered HTML and CSS frontend views. SQLite serves as the primary relational database for all persistent platform data. The AI and NLP layer is implemented using spaCy for named entity recognition and CV skill extraction, with lightweight sentence embeddings supporting semantic similarity computation for job matching. BeautifulSoup4 and Selenium are used for web scraping and dynamic job data collection. The backend development environment is managed using a Python virtual environment to isolate dependencies and ensure consistent package versioning across the development workflow.

1.1.4. System Overview

The AI-Powered Career Assistant Platform operates as a layered client-server system comprising core architectural layers: Presentation, Application, AI/ML, Data, and External Services. Users interact with the system through Django-rendered HTML templates that communicate with the backend through standard HTTP requests and form submissions. The backend handles all intelligent processing tasks within the same application layer, including CV analysis, job matching, and interview evaluation, persisting all data to an SQLite database. Django views manage job scraping workflows using Beautiful Soup and Selenium to collect job listings from external portals, while spaCy powers all NLP operations, including skill extraction and semantic similarity computation for job matching.

1.1.5. Outcome of the Project

The AI-Powered Career Assistant Platform delivers a fully functional, tested, and documented web-based career support ecosystem comprising ten integrated modules. The platform demonstrates the practical application of natural language processing, spaCy-based skill extraction, semantic similarity computation for job matching, and full-stack web development within a unified Django-based system. Upon completion, the project provides students and graduates with an intelligent, centralized platform that meaningfully improves career readiness, reduces job search effort, and increases employability outcomes through data-driven, personalized

career support.

1.2. Relevance to Course Modules

The AI-Powered Career Assistant Platform draws directly upon knowledge acquired across multiple core courses studied throughout the Bachelor of Science in Software Engineering program at COMSATS University Islamabad, Sahiwal Campus. Every stage of the system, from requirements elicitation and architectural design through implementation, integration, and testing, reflects concepts introduced and developed during the degree. The following subheadings explain the specific contribution of each major course module to the project.

1.2.1. Software Engineering

Software Engineering provided the foundational framework governing the entire development lifecycle of the AI-Powered Career Assistant Platform. Concepts, including requirements elicitation, functional and non-functional requirement specification, use case modeling, system architecture design, and SDLC methodology selection, were applied directly throughout the project. The IEEE 830-compliant Software Requirements Specification produced during the requirements phase, the structured UML diagrams developed during system design, and the systematic testing strategy executed across unit, functional, integration, and automated testing levels all reflect core Software Engineering principles applied in practice. The course's coverage of software quality attributes, including performance, reliability, security, scalability, and maintainability, directly informed the non-functional requirements defined for the platform, ensuring that the system is built to professional standards.

1.2.2. Database Systems

Database Systems provided the knowledge required to design and implement the platform's SQLite relational schema, which comprises ten primary entities, including Users, User Profiles, CV Uploads, Job Listings, Applications, Forum Posts, Forum Replies, Notifications, Interview Sessions, and Audit Logs. Core database concepts, including normalization, indexing, query optimization, referential integrity, and data retention policy design, were applied through Django's ORM layer to ensure that sensitive user and career data is stored efficiently and retrieved within acceptable performance thresholds. Database security concepts informed the implementation of

model-level access controls and Django's built-in authentication mechanisms, preventing cross-user

1.2.3. Web Engineering and Internet Programming

Object-oriented programming contributed significantly to the structural organization of the platform's backend codebase. The system's class architecture, comprising nineteen classes including Application, Authentication Manager, CV Analyzer, Job Scraper, Auto Apply Manager, Mock Interview Module, and their associated model and AI/ML support classes, reflects core OOP principles including encapsulation, abstraction, single responsibility, and separation of concerns. The clean separation between business logic classes and Django model classes, and the modular organization of views and utility functions across Django application modules, demonstrate the practical application of OOP design principles within a well-structured Django-based system.

1.2.4. Artificial Intelligence and Machine Learning

Artificial Intelligence and Machine Learning coursework is directly applied through the integration of an NLP layer within the platform. The spaCy pipeline is used for named entity recognition, skill extraction, and qualification identification from CV text, while lightweight sentence embeddings support semantic similarity computation, powering job match scoring, chatbot response generation, and mock interview answer evaluation. Concepts including natural language processing, vector space models, and semantic similarity informed the selection, integration, and application of these models within the Django-based system.

1.2.5. Object-Oriented Programming

Object-oriented programming contributed significantly to the structural organization of the platform's backend codebase. The system's class architecture, comprising nineteen classes including Application, Authentication Manager, CV Analyzer, Job Scraper, Auto Apply Manager, Mock Interview Module, and their associated repository and AI/ML support classes, reflects core OOP principles including encapsulation, abstraction, single responsibility, and dependency injection. The clean separation between business logic classes and Django model classes, and the modular organization of views and utility functions across Django application modules, demonstrate the practical application of OOP design principles within a well-structured Django-

based system.

1.2.6. Data Structures and Algorithms

Data Structures and Algorithms informed the implementation of several core processing operations within the platform. The CV skill extraction pipeline applies spaCy-based token classification and set-based gap detection algorithms. The job match scoring algorithm computes semantic similarity scores between CV content and job descriptions using sentence embeddings and cosine similarity. The mock interview evaluation algorithm applies weighted averaging across question-level semantic similarity scores to compute session performance bands. The forum gamification system maintains a sorted leaderboard updated after each point-awarding event. Each of these operations reflects the application of algorithmic thinking and efficient data handling learned through this course.

1.2.7. Human-Computer Interaction

Human Computer Interaction guided the design of all user-facing interfaces within the Django-rendered HTML and CSS frontend. Nielsen's usability heuristics, WCAG 2.1 Level AA accessibility standards, and principles of simplicity, visual consistency, and intuitive navigation were applied across all ten module screens. The interface follows a consistent layout system, uses the Inter typeface for readability, and implements ARIA labels, 4.5:1 minimum contrast ratios, and full keyboard navigation support. The onboarding flow is designed to be complete within 10 minutes without external guidance, reflecting the course's emphasis on learnability and user-centered design.

1.2.8. Information Security

Information Security played a critical role in the design and implementation of the platform's security layer. Concepts including authentication, authorization, and secure data handling were implemented through Django's built-in session-based authentication, bcrypt password hashing via Django's authentication framework, role-based access control restricting access to admin endpoints to verified administrators through Django's built-in permissions system, and account logout mechanisms triggered after consecutive failed login attempts. Cross-site request forgery protection is enforced through Django's built-in CSRF middleware, and all administrative actions

are recorded in an audit log to maintain platform accountability and data integrity.

1.2.9. Distributed Computing and Computer Networks

Computer Networks informed the design of the platform's inter-component communication architecture and request handling workflows. Concepts such as HTTP request-response cycles, network latency, and client-server communication informed the design of Django's view layer, which handles all synchronous user requests, including CV analysis, job scraping, and auto-apply submissions, within a single application process. Background operations such as job scraping via BeautifulSoup and Selenium are triggered through Django views and management commands, ensuring clean separation between user-facing interactions and long-running processing tasks.

1.2.10. Final Year Project I and II

The Final Year Project I and II courses provided the academic structure, milestone framework, and supervisory guidance under which the AI-Powered Career Assistant Platform was planned, designed, implemented, and evaluated. These courses shaped the systematic approach to project planning, progress documentation, design review, mid-evaluation, and final demonstration that governed the project throughout its lifecycle. The discipline of structured milestone delivery, continuous supervisor feedback integration, and formal academic documentation developed through these courses is reflected throughout the organization and in the completeness of this report.

The AI-Powered Career Assistant Platform is, therefore, a complete applied demonstration of the entire BSE degree, integrating knowledge from programming, artificial intelligence, database design, web engineering, security, distributed systems, and human-computer interaction into a single, coherent, and functional software system.

1.3. Project Background

The idea behind the AI-Powered Career Assistant Platform originates from the increasing difficulty faced by students and graduates in Pakistan when attempting to transition from academic study into professional employment. Traditional approaches to job searching require candidates to manually browse multiple job portals, individually tailor CVs for each application, track

submission history across platforms, and independently identify and develop the skills demanded by their target industries, all without personalized, objective, or intelligent assistance. This manual, fragmented approach is time-consuming, error-prone, and heavily dependent on the candidate's own ability to assess their professional strengths and weaknesses, a task that most recent graduates are poorly equipped to perform without external guidance.

The project idea was developed further through analysis of the specific tools and platforms currently available to Pakistani job seekers. Existing platforms, including LinkedIn, Indeed, and Rozee.pk, provide job listings and basic profile functionality but offer no CV quality analysis, no skill gap detection, no automated application submission, and no interview preparation capabilities. Candidates using these platforms must manually filter through large volumes of listings without any intelligent matching, submit CVs without any objective feedback on their quality or relevance, and prepare for interviews without access to structured, domain-specific practice sessions. The gap between what these platforms offer and what a job seeker genuinely needs to succeed in a competitive application process provided the central motivation for developing a unified, intelligent career assistant.

The project was further motivated by the growing availability and accessibility of powerful natural language processing and transformer-based AI models that can be deployed within production-grade web applications at low cost. The development of pre-trained models such as spaCy and lightweight sentence embedding libraries has made sophisticated NLP capabilities, including named entity recognition, semantic similarity computation, and skill extraction, accessible to application developers without requiring specialized machine learning infrastructure. This accessibility created an opportunity to build an intelligent career support platform that leverages these models to deliver capabilities previously available only in expensive enterprise recruitment tools.

The need for automated job application submission also emerged from observation of the significant time investment required to apply for jobs individually across multiple portals. A candidate actively seeking employment may need to submit dozens of applications per week to achieve a meaningful response rate, each requiring manual form completion, document upload, and portal-specific formatting. The Auto-Apply module of the platform was conceived to eliminate

this burden by automatically identifying listings meeting the user's match threshold and submitting applications through Selenium-based form automation triggered via Django views, allowing candidates to focus their active time on interview preparation and skill development rather than repetitive administrative tasks.

Overall, the AI-Powered Career Assistant Platform is founded on the conviction that intelligent automation and AI-driven personalization can meaningfully democratize access to high-quality career support, enabling students and graduates across Pakistan to compete more effectively in professional job markets regardless of their access to institutional career services or professional mentorship networks.

1.4. Literature Review

The reviewed literature demonstrates that while several career support and job discovery platforms exist, each addresses only a subset of the capabilities required for comprehensive, intelligent career assistance. To clearly highlight the gaps in existing solutions and the specific advantages offered by the AI-Powered Career Assistant Platform, the following table compares six well-known platforms against the proposed system across three dimensions: platform name, identified weakness, and the solution provided by the AI-Powered Career Assistant Platform.

Table 1: Comparison of Existing Career Platforms and AI-Powered Career Assistant Platform

Platform Name	Weakness	Proposed Project Solution
LinkedIn	Provides job listings and profile building but offers no NLP-based CV analysis, no skill gap detection, no automated application submission, and no interview preparation. Premium features are inaccessible to most students due to the subscription	The platform delivers free CV analysis with NLP-based skill extraction and gap detection, automated job match scoring, auto-apply submission, and an adaptive mock interview module, all without a subscription barrier.

	cost.	
Indeed	Aggregates listings from thousands of sources but provides no personalized CV scoring, no semantic match-based job ranking, no career chatbot, and no learning resource integration. Recommendation logic is keyword-based and not tailored to individual skill profiles.	The platform applies Sentence Transformer-based cosine similarity scoring to rank listings against each user's skill profile and integrates a career chatbot and Resource Hub for personalized guidance and learning.
Rozee.pk	Serves the Pakistani job market specifically, but provides no AI-driven features, no CV analysis, no automation, and no interview preparation tools, limiting its utility to a basic job board for local listings.	The platform scrapes and ranks Rozee.pk listings alongside LinkedIn and Indeed, enriching local job data with intelligent match scoring, auto-apply automation, and interview preparation capabilities.

Pramp / Interviewing.io	Provides peer-to-peer and AI-assisted mock interview services but operates entirely in isolation from job searching, CV analysis, and career development, requiring separate accounts and workflows.	The Mock Interview Module is fully integrated within the platform alongside CV analysis, job search, and chatbot modules, enabling users to move seamlessly between career support functions within a single system.
Coursera / Udemy	Provides learning resources across a wide range of skills but does not integrate with a user's CV data or job market requirements, requiring users to independently identify which courses are relevant to their career goals.	The Resource Hub generates personalized learning recommendations directly from skill gaps identified during CV analysis, linking detected deficiencies to targeted Coursera and Udemy courses automatically.
Traditional Manual Job Search	Requires candidates to manually browse portals, tailor CVs, track applications, and prepare for interviews across multiple disconnected platforms with no intelligent assistance, resulting in high time investment and low personalization.	The platform automates job discovery, CV analysis, application submission, and interview preparation within a single intelligent ecosystem, reducing manual effort and delivering personalized guidance at each stage of the job search process.

1.5. Analysis from Literature Review

The analysis of the reviewed literature and platform comparisons demonstrates how the AI-Powered Career Assistant Platform aligns with global trends in intelligent career support technology and addresses the specific gaps present in platforms currently available to Pakistani job seekers. The literature emphasizes the need for centralized career management, intelligent CV assessment, semantic job matching, and structured interview preparation, all of which form the core feature set of the proposed platform. The following subheadings examine the alignment between the reviewed literature and the platform's design across six key dimensions.

1.5.1. Alignment with Digital Career Support Trends

Research in intelligent recruitment and career support systems consistently highlights the shift from keyword-based job matching toward semantic, embedding-based approaches that better capture the conceptual relationships between candidate skills and job requirements. The AI-Powered Career Assistant Platform reflects this trend by implementing Sentence Transformer-based cosine similarity scoring rather than keyword matching for all job recommendation and CV gap detection operations, placing the platform in alignment with the current state of the art in intelligent job matching as described in the reviewed literature.

1.5.2. Comparison with Existing NLP-Based CV Analysis Research

Studies on automated CV analysis, including the work of Devlin et al. (2019) on BERT and Reimers and Gurevych (2019) on Sentence-BERT, establish that transformer-based models significantly outperform rule-based and keyword-matching approaches in extracting meaningful semantic content from professional documents. The platform applies these findings by deploying spaCy's NER pipeline for skill extraction and lightweight sentence embeddings for semantic similarity scoring, ensuring that CV analysis results reflect a meaningful understanding of the document content rather than simple pattern matching.

1.5.3. Relevance to Job Recommendation and Matching Systems

Research on job recommendation systems by Siting et al. (2012) and Qin et al. (2018) demonstrates that content-based filtering and deep semantic matching approaches produce significantly more

relevant recommendations than collaborative filtering or keyword matching when applied to structured candidate profile data. The platform implements these findings through its cosine similarity-based match scoring pipeline, which encodes both job descriptions and user skill profiles as semantic vectors before computing relevance scores, producing rankings that reflect genuine conceptual alignment between candidate competencies and job requirements.

1.5.4. Integration of Automated Application Submission

The literature identifies manual application submission as one of the most time-intensive components of the job search process, with studies indicating that active job seekers spend a significant proportion of their search time on repetitive administrative tasks rather than skill development or interview preparation. The Auto-Apply module directly addresses this finding by delegating application submission to Selenium-based form automation triggered through Django views, freeing users from manual portal interactions while maintaining full submission logging and duplicate prevention through a 30-day application history check.

1.5.5. Application of Adaptive Assessment Principles

Research by VanLehn (2011) on intelligent tutoring systems establishes that adaptive questioning, where follow-up questions are generated based on the quality of a learner's previous response, produces measurably better learning outcomes than static question sequences. The Mock Interview Module directly applies this principle by evaluating each submitted answer using semantic similarity scoring and generating targeted follow-up questions when response scores fall below 60%, ensuring that users receive more practice precisely in the areas where their understanding is weakest.

1.5.6. Validation of Gamification in Community Engagement

Research by Hamari et al. (2014) confirms through a systematic review that gamification elements, including points, badges, and leaderboards, consistently improve user engagement and sustained participation in digital platforms. The Community Forum module's gamification system, awarding 10 points for posts, 5 points for replies, and 2 points for received likes, with badge tier assignments at 50, 200, and 500 point thresholds, is directly validated by these findings, providing a research-

backed rationale for the inclusion of gamification mechanics as a meaningful engagement driver rather than a superficial feature.

1.5.7. Addressing Limitations of Existing Healthcare Technologies

The literature identifies several consistent shortcomings in existing career support platforms, including fragmentation across multiple tools, absence of personalized CV feedback, lack of structured interview preparation, and no integration between job discovery and skill development functions. The AI-Powered Career Assistant Platform addresses each of these limitations by consolidating CV analysis, job scraping, intelligent matching, auto-apply automation, career mentorship, skill-based learning recommendations, peer collaboration, and adaptive interview simulation within a single, integrated, and freely accessible web platform offering a more complete and cohesive career support solution than any single existing platform reviewed in the literature.

1.6. Methodology and Software Lifecycle

The development of the AI-Powered Career Assistant Platform follows a structured approach that combines modern software engineering practices with a suitable Software Development Life Cycle model. Considering the complexity of the system, which integrates ten functional modules spanning full-stack web development, NLP-based AI processing, distributed task automation, real-time WebSocket communication, and relational database management, a methodology supporting iterative development, continuous testing, and adaptive planning was essential.

This approach ensures that all platform modules are thoroughly developed, rigorously tested, and effectively integrated into a cohesive career support ecosystem.

1.6.1. Methodology

The AI-Powered Career Assistant Platform adopts the Agile software development methodology, which emphasizes iterative delivery, continuous feedback integration, and incremental feature development. Agile is particularly well-suited to this project because the integration of machine learning models, third-party web scrapers, and real-time WebSocket communication introduces a high degree of technical uncertainty that is best managed through short development cycles rather than rigid upfront planning. The methodology supports progressive discovery and resolution of

integration challenges between modules, such as the dependency between CV Analyzer skill gap output and Resource Hub recommendation input, ensuring that issues are identified and addressed early in the development process.

1.6.2. Agile Practices Applied

Agile practices are implemented within the project through sprint-based development cycles of two to three weeks each. Each sprint targets a specific set of platform modules, beginning with Authentication as the foundational module upon which all others depend, and progressing through CV Analyzer, Job Scraper, Auto Apply, AI Chatbot, Resource Hub, Community Forum, Notification Module, Admin Panel, and Mock Interview Module in a sequence determined by inter-module dependency. Sprint planning sessions establish the deliverables and acceptance criteria for each cycle. Sprint reviews involve the demonstration of completed functionality and integration of supervisor feedback before the next sprint commences.

1.6.3. Software Development Life Cycle (SDLC) Model

The project adopts the Scrum model of Agile SDLC, dividing development into time-boxed sprints during which specific modules are planned, developed, tested, and reviewed. The Scrum framework provides transparency through sprint planning, daily progress tracking, and sprint retrospectives, ensuring that development remains on schedule and that technical blockers are identified and resolved promptly. The overall project lifecycle follows sequential phases of Requirements Elicitation and Documentation, System Architecture Design, Iterative Module Development, Integration Testing, User Acceptance Testing, and Final Deployment.

1.6.4. Product Backlog Management

The project maintains a structured product backlog comprising all functional requirements across the ten platform modules, organized by module priority and inter-module dependency. Each backlog item is defined with an acceptance criterion, estimated effort, and dependency mapping. Items are selected from the backlog into sprint cycles based on priority and dependency order, ensuring that foundational modules are completed and tested before dependent modules begin development. The backlog is reviewed and reprioritized at the beginning of each sprint based on supervisor feedback and evolving technical understanding.

1.6.5. Incremental Delivery

The platform is designed and built to support incremental delivery, meaning that each completed sprint produces a functional, independently testable increment of the system rather than a partial or non-functional component. For example, the Authentication module delivered at the end of Sprint 1 is fully functional and tested before CV Analyzer development begins in Sprint 2. This incremental approach ensures the continuous availability of a working system state throughout development and reduces integration risk at the project's final stages.

1.6.6. Rationale behind Selected Methodology

The Agile methodology was selected over traditional waterfall approaches for several reasons specific to the nature of this project. The integration of machine learning models, third-party scrapers, and real-time communication channels introduces technical uncertainties that make comprehensive upfront planning impractical. Agile's iterative cycles allow the team to validate technical assumptions, resolve integration challenges, and incorporate supervisor feedback progressively rather than discovering problems late in a sequential development process.

The two-member development team also benefits from Agile's lightweight process overhead, enabling rapid decision-making without the administrative burden of heavyweight formal methodologies.

1.6.7. Development Tools and Environment

The development environment comprises Visual Studio Code as the primary code editor, GitHub for version control and collaborative development, and Postman for API endpoint testing and validation. DB Browser for SQLite serves as the database management interface for inspecting and querying the SQLite development database. Python virtual environments manage all backend dependencies and ensure consistent package versioning across the development workflow. Environment-specific configuration variables, including database credentials, secret keys, and third-party API settings, are managed through .env files loaded via Python's python-dotenv library, ensuring clean separation between configuration and source code across development environments.

1.6.8. Testing and Quality Assurance

Quality assurance is integrated throughout the development lifecycle rather than performed exclusively at the project's end. Unit tests are written alongside each module using Django's built-in testing framework, and are executed at the conclusion of each sprint. Functional and integration tests are performed after each inter-module integration milestone. Automated testing using Django's test client is applied across all critical views and form submissions. The combined testing strategy confirms that all functional and non-functional requirements defined in the SRS are met.

1.6.9. Advantages of the Selected Methodology and SDLC

The Agile Scrum methodology offers several concrete advantages for this project. The iterative sprint structure ensures that each module is independently functional before integration begins, reducing the complexity of debugging across module boundaries.

Continuous supervisor feedback integration ensures that the system remains aligned with academic requirements and real-world career support needs throughout development. The sprint-based delivery schedule creates natural project milestones that facilitate academic progress reporting and evaluation. The methodology's emphasis on working software over comprehensive documentation at each iteration ensures that the project delivers a demonstrable, functional system at every stage of the development lifecycle.

Chapter 2

PROBLEM DEFINITIO

2. Problem Definition

Fresh graduates and early-career professionals in Pakistan face significant and compounding difficulties when attempting to enter competitive job markets without adequate tools, personalized guidance, or centralized resources. The job search process is deeply fragmented, requiring candidates to manually browse LinkedIn, Indeed, and Rozee.pk independently, tailor CVs for individual applications without any objective quality feedback, track submission history across multiple platforms, and prepare for technical interviews without access to structured, domain-specific practice environments.

This manual approach is not only time-consuming but also highly inefficient, as most candidates lack the expertise to objectively evaluate the quality and relevance of their own CVs, identify their skill gaps relative to current industry demand, or simulate the question formats and evaluation criteria applied during real technical interviews. Pakistan produces hundreds of thousands of university graduates annually across science, technology, engineering, and mathematics disciplines, yet graduate unemployment and underemployment rates remain persistently high. A significant contributing factor to this outcome is not a shortage of technically capable candidates but rather a systemic absence of the tools, infrastructure, and personalized support that would enable those candidates to present their capabilities effectively, identify and close their skill gaps efficiently, and navigate the application and interview process with confidence.

In the absence of institutional career services at most Pakistani universities and the financial inaccessibility of professional career coaching, the majority of graduates enter the job market without any structured support beyond their academic qualifications. A critical and largely unaddressed problem within this space is the absence of tools capable of intelligently connecting a candidate's existing skill profile to the specific requirements of available job opportunities.

Most job portals apply rudimentary keyword-based filtering that fails to capture the semantic relationships between a candidate's competencies and a job description's requirements, resulting in poor match quality and high volumes of irrelevant listings that candidates must manually evaluate. Candidates frequently submit poorly optimized resumes that fail to pass automated Applicant Tracking Systems used by recruiters, reducing their visibility regardless of their actual

technical capability. The repetitive administrative burden of manually searching, tailoring, and submitting applications across multiple portals consumes time that could otherwise be invested in skill development and interview preparation, the activities that would most meaningfully improve a candidate's employment prospects.

2.1 Problem Statement

Fresh graduates and early-career professionals in Pakistan face significant and compounding difficulties when attempting to enter competitive job markets without adequate tools, personalized guidance, or centralized resources. The job search process is deeply fragmented, requiring candidates to manually browse LinkedIn, Indeed, and Rozee.pk independently, tailor CVs for individual applications without any objective quality feedback, track submission history across multiple platforms, and prepare for technical interviews without access to structured, domain-specific practice environments. This manual approach is not only time-consuming but also highly inefficient, as most candidates lack the expertise to objectively evaluate the quality and relevance of their own CVs, identify their skill gaps relative to current industry demand, or simulate the question formats and evaluation criteria applied during real technical interviews.

A critical and largely unaddressed problem within this space is the absence of tools capable of intelligently connecting a candidate's existing skill profile to the specific requirements of available job opportunities. Most job portals apply rudimentary keyword-based filtering that fails to capture the semantic relationships between a candidate's competencies and a job description's requirements, resulting in poor match quality and high volumes of irrelevant listings that candidates must manually evaluate. Candidates frequently submit poorly optimized resumes that fail to pass automated Applicant Tracking Systems used by recruiters, reducing their visibility regardless of their actual technical capability.

2.2 Deliverables and Development Requirements

2.2.1 Deliverables

The AI-Powered Career Assistant Platform delivers the following upon project completion:

The system includes a fully functional, browser-accessible full-stack web application comprising

ten integrated and independently testable career support modules. Built using Django 4.x with Django's templating engine for server-side rendered HTML and CSS views, it ensures a responsive and consistent user experience. The web application serves as the primary interface through which registered users access CV analysis, job search, AI chatbot, mock interview, community forum, and resource hub features.

The platform provides a structured backend built with Django 4.x, handling all functionality through Django views and URL routing. It implements session-based authentication, role-based access control via Django's permissions system, and request validation across all views. All module interactions operate through standard HTTP request-response cycles, connecting the frontend templates with the spaCy NLP pipeline, BeautifulSoup and Selenium scraping layer, and SQLite database.

The system incorporates a server-side AI and NLP processing pipeline utilizing spaCy's named entity recognition for CV skill extraction and lightweight sentence embeddings for semantic similarity computation and cosine similarity scoring. This pipeline powers CV quality scoring, skill gap detection, job match percentage calculation, AI chatbot response generation, and mock interview answer evaluation across intelligent modules.

An asynchronous task processing infrastructure powered by Celery and Redis manages background operations, including scheduled job scraping from LinkedIn, Indeed, and Rozee.pk every six hours, asynchronous CV analysis dispatch, auto-apply submission execution, and weekly email digest generation every Sunday at 8 AM. The task queue applies a three-retry exponential backoff policy for failed tasks and provides monitoring visibility through Celery Flower.

A dedicated administrator interface provides comprehensive governance capabilities, including user account management, forum content moderation, job scraper health monitoring, auto-apply task oversight, platform analytics, and audit log export in CSV format. Role-based access control ensures that all administrative functions are accessible exclusively to verified system administrators.

2.2.2 Development Requirements

The development requirements define the precise technical environment, framework versions, and system dependencies necessary to build, run, and deploy the AI-Powered Career Assistant Platform consistently across development and production settings.

The technical requirements define the tools, frameworks, and infrastructure components necessary to develop and operate the platform across all architectural layers.

- **Backend Development:** Python 3.x within a configured virtual environment for dependency management. Django 4.x for view handling, URL routing, and business logic implementation, with Django's built-in authentication framework supporting session-based authentication and role-based access control.
- **AI and NLP Layer:** spaCy for named entity recognition, skill extraction, and CV text analysis, along with lightweight sentence embeddings for semantic similarity and cosine similarity scoring to support job match calculation and interview answer evaluation.
- **Web Scraping:** BeautifulSoup4 and Selenium WebDriver for scraping job listings from LinkedIn, Indeed, and Rozee.pk. PyMuPDF and python-docx for text extraction from uploaded PDF and DOCX CV files.
- **Frontend Development:** Django's templating engine for server-side rendered HTML and CSS views, with Vanilla JavaScript enabling client-side interactivity and dynamic behavior across modules.
- **Database:** SQLite as the primary relational database, managed through Django's ORM for abstraction and query handling, with DB Browser for SQLite for database administration and inspection.
- **Server and Deployment:** Django's built-in development server for local execution and python-dotenv for environment-specific configuration management using .env files.
- **Development Tools:** Visual Studio Code as the primary code editor, GitHub for version control and collaboration, Postman for endpoint testing and validation, and DB Browser for SQLite for database inspection and query management.

2.3 Current System

Several platforms currently exist that partially address the career support needs of job seekers, but none provide a comprehensive, centralized, and intelligent solution comparable to the proposed platform. The fragmented landscape of existing tools forces candidates to maintain multiple accounts, manage disconnected workflows, and independently synthesize insights from separate systems, a cognitive and administrative burden that most graduates are poorly positioned to manage effectively alongside their academic commitments.

The following subsections examine the major categories of existing systems, their specific limitations, and the problems that those limitations create for Pakistani graduates entering the professional job market.

2.3.1 Manual and Fragmented Job Search Process

The predominant approach to job searching among Pakistani graduates remains manual and highly fragmented. Candidates independently browse multiple job portals, manually evaluate listing relevance without any intelligent scoring, individually compose and submit applications for each position, and track submission history through personal spreadsheets or memory. This approach demands a disproportionate investment of time in repetitive administrative tasks, leaving candidates with insufficient time for the skill development and interview preparation activities that would meaningfully improve their employment outcomes. The absence of any centralized tracking or automation means that candidates frequently miss application deadlines, submit duplicate applications, or lose track of their submission history entirely. Research consistently indicates that active job seekers spend a majority of their search time on administrative tasks rather than value-adding activities, and the manual, fragmented approach prevalent among Pakistani graduates exemplifies this inefficiency at scale.

2.3.2 Limitations of Existing Job Portals

Existing job portals, including LinkedIn, Indeed, and Rozee.pk, serve as listing aggregators and basic profile platforms but fall significantly short of providing intelligent career support. LinkedIn's job matching relies on keyword-based profile and listing comparison rather than semantic skill analysis, producing recommendations of highly variable relevance. Its premium career features, including resume review, InMail credits, and advanced job insights, are locked

behind subscription tiers priced beyond the reach of most Pakistani students and graduates. Indeed provides no personalized CV scoring, no semantically ranked job recommendations, and no career development features of any kind. Rozee.pk, while serving the Pakistani market specifically, functions as a basic job board with no AI-driven features, no CV analysis, no skill gap detection, and no interview preparation capabilities, offering no meaningful differentiation from a fully manual job search approach.

2.3.3 Absence of Intelligent CV Analysis Tools

No existing platform accessible to Pakistani graduates provides NLP-based CV quality analysis that objectively evaluates a submitted CV against industry standards, extracts and maps the candidate's skills to a structured taxonomy, computes a weighted quality score across multiple dimensions, and identifies specific skill gaps relative to target role requirements. Free online CV checkers typically apply simple keyword matching against a fixed list of terms, producing generic and often inaccurate feedback that fails to reflect the semantic content of the candidate's professional experience. Professional CV review services are expensive, inaccessible to most students, and provide one-time feedback rather than continuous, updated analysis as the candidate's skills and the job market evolve. This absence of accessible, intelligent CV analysis leaves the majority of Pakistani graduates without objective, actionable insight into the quality and relevance of their professional profile at the most critical stage of their career development.

2.3.4 Lack of Automated Job Application Submission

The process of applying to job listings manually across multiple portals represents one of the most time-intensive and repetitive components of the job search process. A candidate actively seeking employment may need to submit dozens of applications per week across LinkedIn, Indeed, and Rozee.pk to achieve a meaningful recruiter response rate, each requiring individual form completion, document upload, and portal-specific formatting. No existing platform accessible to Pakistani graduates provides automated application submission that identifies relevant listings, verifies match eligibility against a user-defined threshold, prevents duplicate submissions, and maintains a comprehensive application history — all without requiring active user involvement for each submission. This absence of automation forces candidates to dedicate significant time to administrative application tasks at the expense of skill development and interview preparation

activities that would more meaningfully improve their employment outcomes.

2.3.5 Lack of Integrated Interview Preparation

Mock interview tools such as Pramp and Interviewing.io provide peer-to-peer and AI-assisted interview practice but operate in complete isolation from job searching, CV analysis, and career development workflows, requiring separate accounts, separate workflows, and significant independent time investment. These platforms are predominantly oriented toward software engineering roles in Western job markets and do not address the specific domains, question formats, difficulty levels, and evaluation criteria relevant to Pakistani technical recruitment environments. The absence of an integrated, adaptive interview preparation module within a broader career support ecosystem means that candidates must context-switch between multiple platforms and independently synthesize connections between their CV gaps, learning resource engagement, and interview performance in order to form a coherent and actionable picture of their overall career readiness.

2.3.6 Disconnection Between Skill Development and Career Planning

Existing learning platforms, including Coursera and Udemy, provide extensive libraries of learning resources across a wide range of technical and professional skills, but offer no integration with a user's CV data, job market demand, or application history. Candidates must independently identify which skills are demanded by their target roles, assess which of those skills they currently lack relative to industry benchmarks, and manually search for relevant learning resources, a process that requires significant self-awareness and industry knowledge that most recent graduates in Pakistan do not yet possess. The absence of a data-driven connection between CV-identified skill gaps and targeted, role-specific learning recommendations means that most candidates invest their limited development time in courses that may not directly address the specific competency gaps most likely to limit their employability in their chosen field.

2.3.7 Absence of Community and Peer Support Infrastructure

Beyond the technical tools and resources required for job searching, CV optimization, and interview preparation, Pakistani graduates also lack access to structured peer communities where they can share experiences, seek advice, discuss industry trends, and learn from the career journeys of others navigating similar challenges. Professional networking communities on LinkedIn are

predominantly populated by experienced professionals and do not provide an accessible, supportive environment for students and recent graduates seeking entry-level guidance. No existing platform combines a career-focused peer community with gamification mechanics that incentivize meaningful contribution and sustained engagement, leaving graduates without a structured social learning environment that complements the individual career support tools available to them.

2.4 Summary

The analysis of existing systems presented in this chapter confirms that no single platform currently available to Pakistani graduates adequately addresses the full spectrum of career support needs faced during the transition from academic study to professional employment. Each existing solution addresses at most one or two dimensions of the problem. Job portals provide listings without intelligence, CV checkers provide feedback without integration, interview platforms provide practice without context, and learning platforms provide resources without personalization.

The AI-Powered Career Assistant Platform directly addresses every limitation identified across all seven subsections by consolidating intelligent CV analysis, automated job discovery and match scoring, auto-apply submission with duplicate prevention, real-time AI career mentorship, adaptive mock interview simulation, data-driven learning resource recommendations, and a gamified peer community into a single, integrated, and freely accessible web platform. The platform represents a comprehensive and cohesive response to the systemic career readiness challenges documented in this chapter, designed specifically to serve the needs of students and graduates navigating Pakistan's competitive professional employment market.

Chapter 3

REQUIREMENT ANALYSIS

3. Requirement Analysis

The Requirement Analysis chapter provides a comprehensive and structured understanding of what the proposed system must accomplish in order to function effectively as a career support platform. It serves as a foundation for translating user expectations into clear technical requirements by identifying system behavior, user interactions, and the specific tasks the system must perform. This chapter outlines how the AI-Powered Career Assistant Platform is expected to operate, detailing both functional and non-functional requirements while defining the roles and responsibilities of each actor involved, such as registered users and administrators. Using the use case diagrams presented in this chapter as a reference, the requirements are organized into well-defined sections that guide the design, development, and validation processes. Through this analysis, the development team gains a precise understanding of the system's objectives, constraints, and performance expectations, ensuring that the final implementation aligns with the needs of all stakeholders and effectively addresses the limitations of the current fragmented career support landscape.

3.1 Introduction to Requirement Analysis

Requirement Analysis is a critical phase in the software development lifecycle, serving as the foundation upon which the entire system is designed, implemented, and evaluated. For the AI-Powered Career Assistant Platform, this phase ensures that all functional expectations, performance needs, and operational constraints are clearly understood before development begins. The purpose of conducting requirement analysis is to bridge the gap between user needs and technical implementation, ensuring the final system effectively resolves existing career support challenges such as fragmented job searching, absence of intelligent CV analysis, lack of automated application submission, and disconnected interview preparation tools. This chapter outlines the detailed requirements of the platform, describing how each system component interacts with users, how data flows between modules, and what outcomes are expected from every interaction across all ten platform modules.

3.2 System Overview

The AI-Powered Career Assistant Platform is designed as an integrated digital ecosystem that connects students, graduates, and early-career professionals with intelligent career support tools through a single, unified web-based interface. The system enables registered users to upload and analyze their CVs using NLP-based pipelines, discover and filter job listings scraped from LinkedIn, Indeed, and Rozee.pk, automatically submit applications for listings meeting their defined match threshold, receive real-time career guidance through an AI chatbot, explore personalized learning resource recommendations linked to their identified skill gaps, engage with peers through a gamified community forum, and build interview confidence through an adaptive mock interview simulator. The Administrator oversees the entire platform by managing user accounts, moderating forum content, monitoring scraper health, and reviewing audit logs from a dedicated admin panel.

The use case diagrams presented in this chapter illustrate the relationships between these actors and the system, showing how functionalities such as CV analysis, job matching, auto-apply submission, chatbot interaction, and mock interview sessions are interconnected within the platform's architecture. This overview reflects the system's role as a centralized career support hub that replaces fragmented, manual career management workflows with a streamlined, AI-powered digital solution accessible from any standard web browser.

3.3 Actors in the System

The AI-Powered Career Assistant Platform interacts with two primary categories of users, each performing distinct roles that contribute to the overall functionality of the platform. These users, also known as actors, engage with the system through specific use cases that reflect their responsibilities, permissions, and operational needs. Understanding these actors and their interactions is crucial for defining accurate system requirements and ensuring that the platform delivers a seamless and secure experience to all users.

3.3.1 User

The User is the primary beneficiary of the AI-Powered Career Assistant Platform. Users interact with the platform to register accounts, upload and analyze their CVs, search and filter job listings,

enable automated job application submission, use the AI career chatbot, browse the Resource Hub, participate in the Community Forum, and take mock interview sessions. The system ensures that users can manage their complete career development journey within a single platform without relying on multiple disconnected tools or manual processes.

3.3.2 Administrator

The Administrator oversees the entire AI-Powered Career Assistant Platform. Their responsibilities include user account management, forum content moderation, job scraper health monitoring, auto-apply task oversight, platform analytics review, and audit log management. Administrators handle the verification and suspension of user accounts, remove violating forum content, monitor scraper execution status, and export audit logs in CSV format for compliance purposes. Their admin panel provides tools for monitoring platform health and ensuring that all system operations remain secure, accurate, and aligned with platform policies.

3.4 Use Case Diagrams

Use case diagrams provide a clear and structured representation of how different users interact with the AI-Powered Career Assistant Platform. They help define the system's functional boundaries and illustrate the various operations performed by each actor. Five use case diagrams are presented below, each covering a distinct functional area of the platform to maintain clarity and readability.

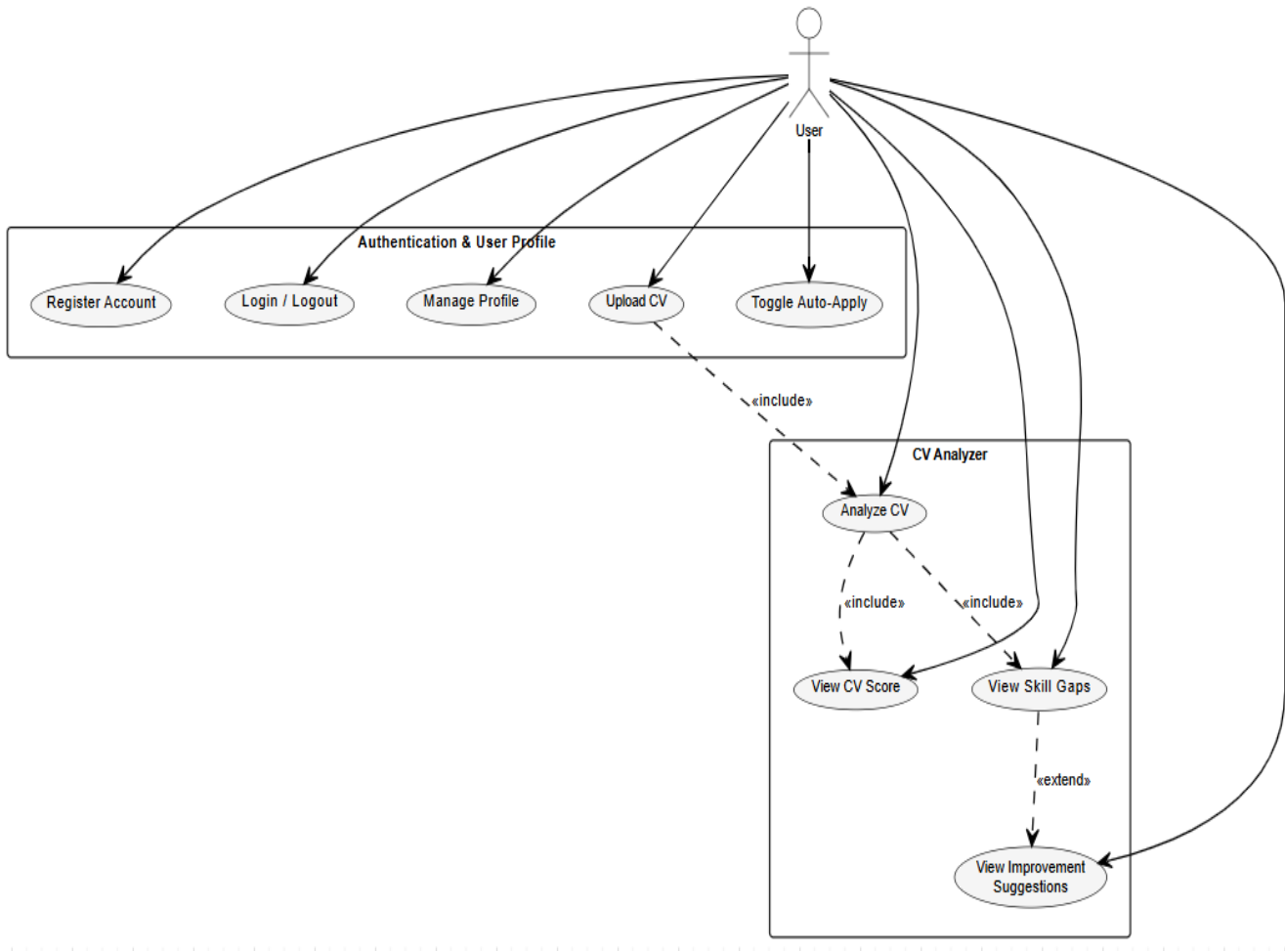


Figure 1:Use Case Diagram - Authentication & CV Analyzer

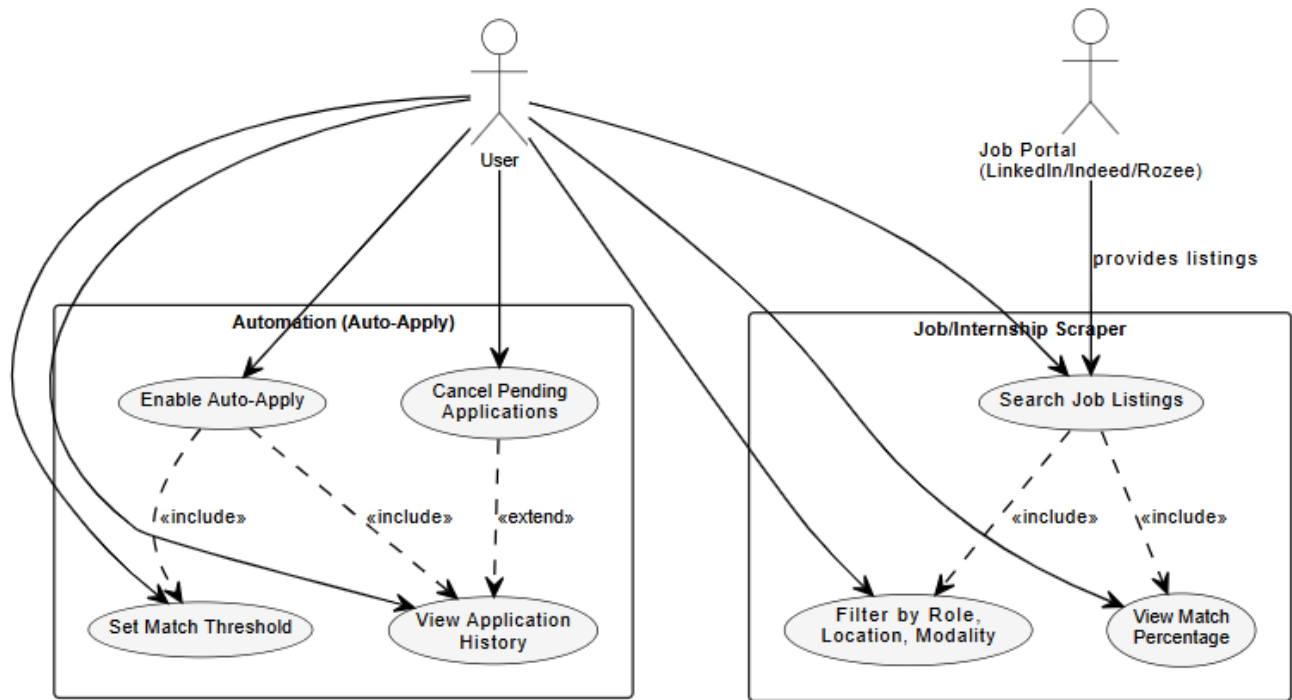


Figure 2: Use Case Diagram - Job Scraper & Auto-Apply

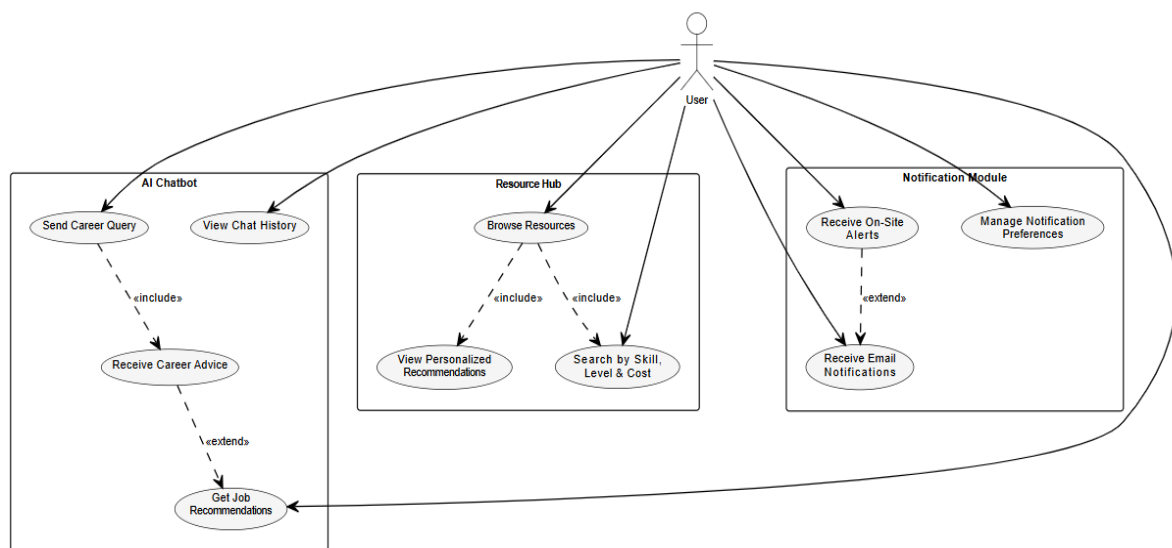


Figure 3: Use Case Diagram - AI Chatbot, Resource Hub & Notifications

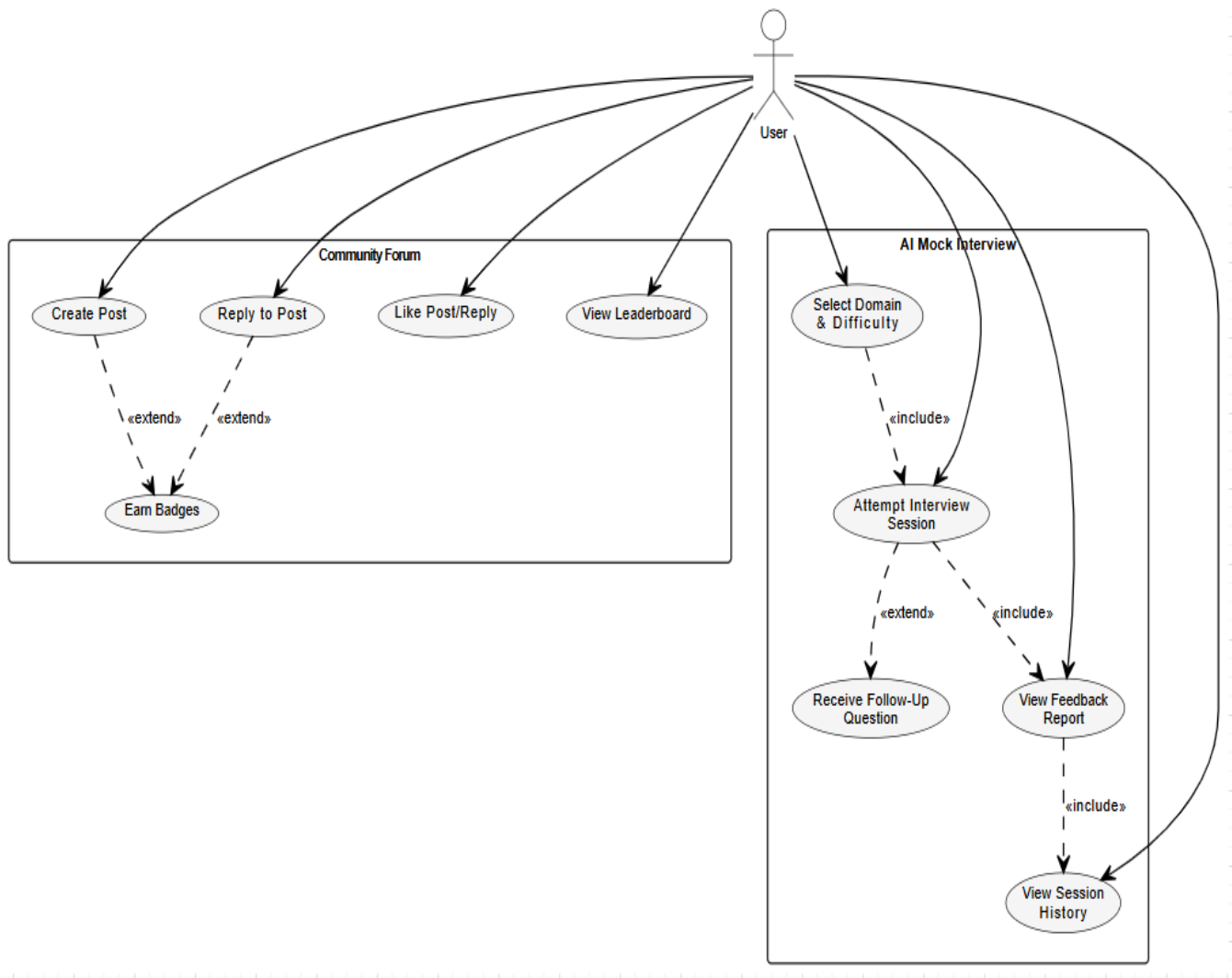


Figure 4: Use Case Diagram - Community Forum & Mock Interview

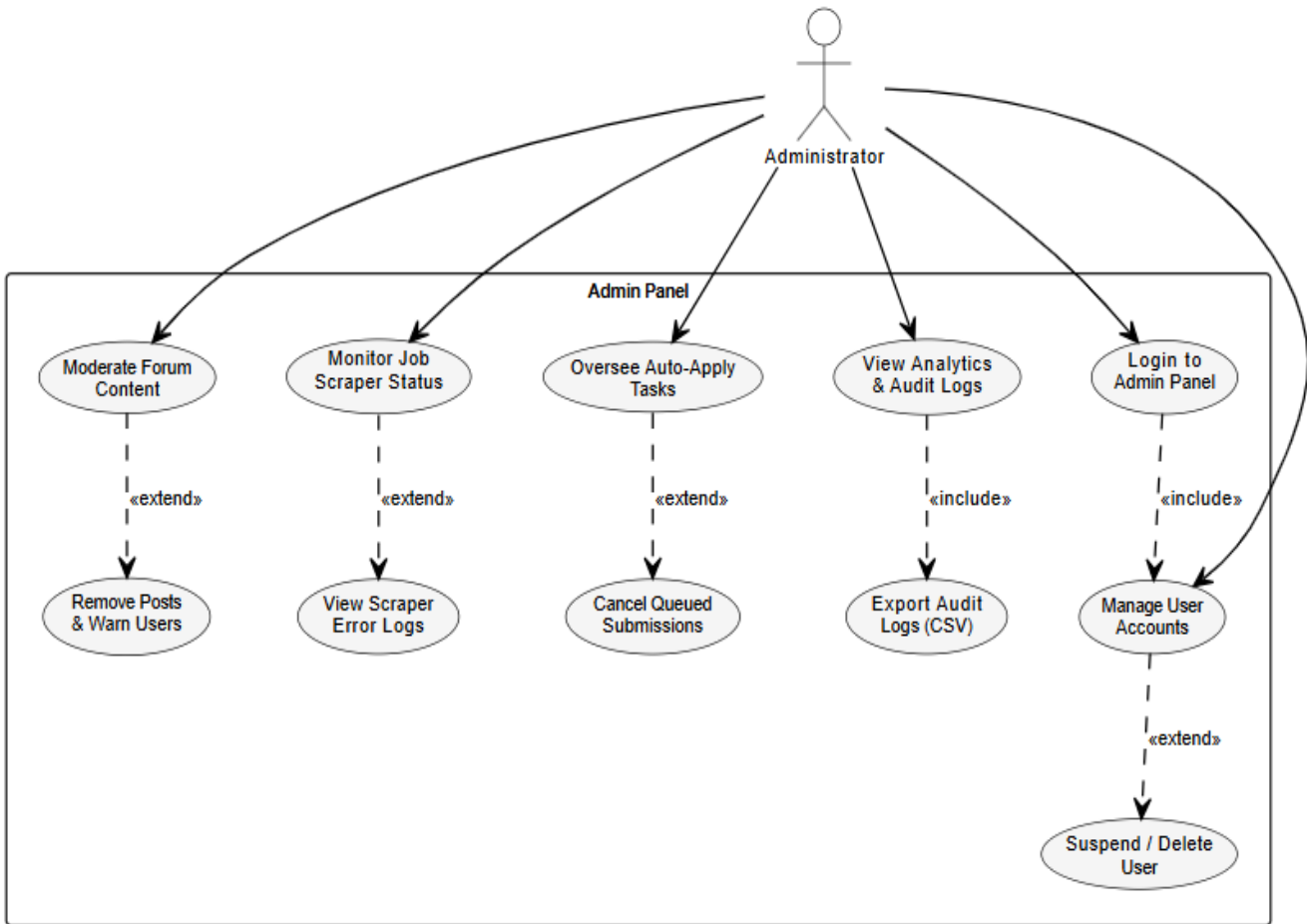


Figure 5: Use Case Diagram - Admin Panel

3.5 Detailed Use Cases

The following tables provide structured descriptions of all major use cases within the AI-Powered Career Assistant Platform. Each table defines the use case name, primary actor, description, trigger, pre-conditions, post-conditions, normal flow, alternative flow, exceptions, and business rules for a specific system interaction. A total of fifteen detailed use case tables are presented, covering all primary functional interactions across the platform's ten modules.

3.5.1 User Registration and Login

The "User Registration and Login" use case covers the two-phase process through which new users create accounts and existing users authenticate on the AI-Powered Career Assistant Platform. This use case is triggered whenever a new user attempts to register or an existing user attempts to log

in. As outlined in Table 3.1, the precondition for registration requires that the user provide a valid, previously unregistered email address. The normal flow involves the user entering their credentials, the system validating input, and upon successful authentication, issuing a JWT access token and redirecting the user to the Dashboard. Alternative flows handle invalid credentials and repeated failed login attempts, while business rules enforce bcrypt password hashing, 60-minute JWT token expiry, and account lockout after five consecutive failed login attempts.

Table 2: Structured description of the User Registration and Login use case

Use Case Attribute	Details
Use Case Name	User Registration and Login
Primary Actor	User
Description	New users register accounts using email and password. Existing users authenticate using valid credentials and receive a JWT token granting access to the Dashboard.
Trigger	A new user attempts to register, or an existing user attempts to log in.
Pre-Conditions	For registration, the email must not already exist in the system. For login, the user must have a registered and active account.
Post-Conditions	A new account is created and stored. An authenticated user is redirected to the Dashboard with a JWT access token issued.
Normal Flow	1. User navigates to registration or login page → 2. User enters credentials → 3. System validates input → 4. System creates account or verifies credentials → 5. JWT token issued → 6. User redirected to Dashboard
Alternative Flow	Invalid credentials return a 401 error. Duplicate email during registration returns a 400 error.
Exceptions	Server error, no internet connection, database unavailability
Business Rules	Passwords must be stored as bcrypt-hashed values. JWT tokens

	expire after 60 minutes. Accounts are locked for 15 minutes after 5 consecutive failed login attempts.
--	--

3.5.2 Manage User Profile

The "Manage User Profile" use case allows authenticated users to update their professional profile information within the platform. This use case is triggered whenever a user wants to update their display name, contact details, declared skills, job role preferences, or Auto-Apply match threshold setting. As outlined in Table 3.2, the precondition requires that the user is authenticated with a valid JWT token. The normal flow involves the user navigating to the profile screen, editing the desired fields, and submitting the changes, which the system validates and persists to the UserProfiles table. Alternative flows handle validation failures, while business rules ensure that updated skill data is immediately applied to subsequent job match scoring operations.

Table 3: Structured description of the Manage User Profile use case

Use Case Attribute	Details
Use Case Name	Manage User Profile
Primary Actors	User
Description	Authenticated users can update their professional profile, including skills, job preferences, and Auto-Apply threshold settings.
Trigger	User wants to update their profile information.
Pre-Conditions	User must be authenticated with a valid JWT token.
Post-Conditions	Profile is updated successfully in the User Profiles table.
Normal Flow	1. Login → 2. Navigate to the profile screen → 3. Edit desired fields → 4. Submit changes → 5. System validates and persists updates
Alternative Flow	Invalid field input returns a field-level validation error and prevents submission.

Exceptions	Server error, expired JWT token, no internet connection.
Business Rules	Updated skill data must be immediately applied to job match scoring. All profile changes must be validated before persistence.

3.5.3 Upload and Analyze CV

The "Upload and Analyze CV" use case enables authenticated users to upload a CV file in PDF or DOCX format and receive a comprehensive NLP-based analysis of their CV quality, extracted skills, and identified skill gaps. This use case is triggered whenever a user uploads a CV file through the CV Analyzer module. As outlined in Table 3.3, the precondition requires that the uploaded file is in a supported format and does not exceed the 5MB size limit. The normal flow involves file validation, asynchronous Celery task dispatch, NLP-based skill extraction and scoring, and return of results to the user dashboard. Alternative flows handle unsupported file formats and oversized files, while business rules enforce asynchronous processing to prevent blocking of synchronous API operations.

Table 4: Structured description of the Upload and Analyze CV use case

Use Case Attribute	Details
Use Case Name	Upload and Analyze CV
Primary Actors	User
Description	Users upload a CV in PDF or DOCX format. The system analyzes the CV using NLP to extract skills, generate a quality score, and detect skill gaps.
Trigger	User uploads a CV file through the CV Analyzer module.
Pre-Conditions	File must be in PDF or DOCX format and must not exceed 5MB. User must be authenticated.
Post-Conditions	CV is stored in the CV Repository. Analysis results, including score, skills, gaps, and suggestions, are displayed on the

	dashboard.
Normal Flow	1. Login → 2. Navigate to CV Analyzer → 3. Upload CV file → 4. System validates file → 5. Celery task dispatched → 6. NLP pipeline extracts skills and generates a score → 7. Results returned and displayed
Alternative Flow	Unsupported format returns "Only PDF and DOCX formats are accepted." File exceeding 5MB returns "File size must not exceed 5MB."
Exceptions	Celery task failure triggers a retry up to 3 times with exponential backoff. Server error, no internet connection.
Business Rules	CV analysis must be processed asynchronously via Celery. Results must be persisted before being returned to the frontend.

3.5.4 Search and Filter Job Listings

The "Search and Filter Job Listings" use case allows authenticated users to search for job listings scraped from LinkedIn, Indeed, and Rozee.pk, applying filters by job role, location, and work modality. This use case is triggered whenever a user navigates to the Job Search module and submits search criteria. As outlined in Table 3.4, the precondition requires that job listings have been scraped and stored in the Job Repository. The normal flow involves the user entering filters, the system querying the repository, and returning a paginated feed of listings ranked by match score. Alternative flows handle empty results, while business rules require that listings older than 30 days are automatically purged from the repository.

Table 5: Structured description of the Search and Filter Job Listings use case.

Use Case Attribute	Details
Use Case Name	Search and Filter Job Listings
Primary Actors	User
Description	Users search and filter scraped job listings by role, location, and

	modality. The system returns listings ranked by computed match score.
Trigger	User navigates to Job Search and submits search criteria.
Pre-Conditions	Job listings must have been scraped and stored in the Job Repository. User must be authenticated.
Post-Conditions	Filtered and ranked job listings are returned and displayed as a paginated job feed with match percentage badges.
Normal Flow	1. Login → 2. Navigate to Job Search → 3. Enter filters → 4. System queries Job Repository → 5. Ranked listings returned and displayed
Alternative Flow	No matching listings returns "No listings found matching your criteria."
Exceptions	Scraper failure, empty Job Repository, server error, no internet connection
Business Rules	Listings older than 30 days must be automatically purged. Match scores must be computed using cosine similarity against the user's skill profile.

3.5.5 Enable Auto-Apply

The "Enable Auto-Apply" use case allows authenticated users to enable automated job application submission for all listings meeting their defined match threshold. This use case is triggered whenever a user enables the Auto-Apply toggle from their profile settings. As outlined in Table 3.5, the precondition requires that the user's profile contains a defined match threshold and that job listings are available in the Job Repository. The normal flow involves the system identifying eligible listings, checking the 30-day duplicate prevention window, enqueueing Celery submission tasks, and notifying the user of each submission. Business rules enforce duplicate prevention and full submission logging.

Table 6:Structured description of the Enable Auto-Apply use case

Use Case Attribute	Details
Use Case Name	Enable Auto-Apply
Primary Actors	User
Description	Users enable automated application submission for listings meeting their defined match threshold. The system submits applications, prevents duplicates, and notifies the user.
Trigger	User enables the Auto-Apply toggle from profile settings.
Pre-Conditions	User must be authenticated. The match threshold must be defined in the user profile. Job listings must be available in the Job Repository.
Post-Conditions	Applications submitted for eligible listings. All attempts are logged in Application History. User is notified of each submission.
Normal Flow	1. Login → 2. Enable Auto-Apply toggle → 3. System identifies threshold-meeting listings → 4. Duplicate check performed → 5. Celery tasks enqueued → 6. Submissions logged → 7. User notified
Alternative Flow	Duplicate listing within a 30-day window is blocked and logged as a duplicate.
Exceptions	Celery task failure triggers 3 retries with exponential backoff. Server error or no internet connection.
Business Rules	Duplicate applications to the same listing within 30 days are strictly prohibited. All submission attempts must be logged with status and timestamp.

3.5.6 Use AI Chatbot

The "Use AI Chatbot" use case enables authenticated users to submit career-related queries through a real-time WebSocket-connected conversational interface and receive AI-generated career advice and job recommendations. This use case is triggered whenever a user opens the chatbot panel and submits a message. As outlined in Table 3.6, the precondition requires that a WebSocket connection be established between the frontend and the Django Channels backend. The normal flow involves query processing by the AChatbot class, TransformerModel response generation, and delivery of the response within 3 seconds. Business rules require conversation context to be maintained within the active session.

Table 7: Structured description of the Use AI Chatbot use case

Use Case Attribute	Details
Use Case Name	Use an AI Chatbot for Career Guidance
Primary Actors	User
Description	Users submit career queries through a real-time WebSocket interface and receive AI-generated career advice and job recommendations within 3 seconds.
Trigger	User opens the chatbot panel and submits a career query.
Pre-Conditions	User must be authenticated. WebSocket connection must be established via WSS /api/chat/.
Post-Conditions	Career advice response is delivered to the user. Conversation history is stored in the Chat History data store.
Normal Flow	1. Login → 2. Open chatbot panel → 3. WebSocket connection established → 4. User submits query → 5. AChatbot processes message → 6. TransformerModel generates response → 7. Response delivered within 3 seconds
Alternative Flow	Lost WebSocket connection triggers automatic reconnection after 5 seconds.

Exceptions	Response generation timeout, server error, WebSocket disconnection
Business Rules	Responses must be delivered within 3 seconds. Conversation context must be maintained within the active session.

3.5.7 Browse Resource Hub

The "Browse Resource Hub" use case allows authenticated users to explore curated learning resources from Coursera and Udemy, with personalized recommendations generated from their CV analysis skill gap results. This use case is triggered whenever a user navigates to the Resource Hub module. As outlined in Table 3.7, the precondition requires that CV analysis has been completed and skill gap data is available. The normal flow involves the system retrieving gap data, generating personalized recommendations, and displaying them categorized by skill, level, and cost. Business rules require that recommendations be directly linked to detected skill gaps.

Table 3.7: Structured description of the Browse Resource Hub use case

Use Case Attribute	Details
Use Case Name	Browse Resource Hub
Primary Actors	User
Description	Users browse personalized learning resource recommendations from Coursera and Udemy, linked to skill gaps identified during CV analysis.
Trigger	User navigates to the Resource Hub module.
Pre-Conditions	CV analysis must have been completed. Skill gap data must be available in the Analysis Results store.
Post-Conditions	Personalized resource recommendations are displayed, categorized by skill, level, and cost.
Normal Flow	1. Login → 2. Navigate to Resource Hub → 3. System retrieves skill gap data → 4. Recommendations generated →

	5. Resources displayed by skill, level, and cost
Alternative Flow	No CV analysis completed prompts the user to upload a CV before accessing personalized recommendations. General resources remain accessible via manual search.
Exceptions	Resource retrieval error, server error, no internet connection
Business Rules	Personalized recommendations must be directly derived from CV analysis skill gap results.

3.5.8 Create Forum Post and Reply

The "Create Forum Post and Reply" use case allows authenticated users to create new discussion posts or reply to existing posts within the Community Forum, earning contribution points and badge tier assignments for each interaction. This use case is triggered whenever a user submits a new post or reply. As outlined in Table 3.8, the precondition requires that the user is authenticated and the submitted content meets minimum length requirements. Business rules enforce the gamification point allocation system and badge tier evaluation after each contribution.

Table 8: Structured description of the Create Forum Post and Reply use case

Use Case Attribute	Details
Use Case Name	Create Forum Post and Reply
Primary Actors	User
Description	Users create posts or reply to existing discussions. The system awards contribution points, evaluates badge eligibility, and updates the leaderboard after each interaction.
Trigger	User submits a new post or reply within the Community Forum.
Pre-Conditions	User must be authenticated. Content must meet the minimum length requirement and comply with platform guidelines.
Post-Conditions	Post or reply is published. The author receives 10 points for a post or 5 points for a reply. Badge tier eligibility is evaluated.

	Leaderboard is updated.
Normal Flow	1. Login → 2. Navigate to Forum → 3. Select category → 4. Submit post or reply → 5. System publishes content → 6. Points awarded → 7. Badge eligibility evaluated → 8. Leaderboard updated
Alternative Flow	Flagged content is held for admin review, and the user is notified.
Exceptions	Server error, content validation failure, no internet connection
Business Rules	10 points awarded per post, 5 points per reply, 2 points per received like. Badge tiers are assigned at 50, 200, and 500 point thresholds.

3.5.9 Take Mock Interview Session

The "Take Mock Interview Session" use case enables authenticated users to initiate an adaptive mock technical interview by selecting a domain and difficulty level. This use case is triggered whenever a user starts a new interview session from the Mock Interview module. As outlined in Table 3.9, the precondition requires that domain and difficulty selections be made before session initiation. The normal flow involves AI question generation, user response submission, semantic similarity-based evaluation, adaptive follow-up question generation for low scores, and feedback report generation upon session completion. Business rules enforce follow-up question generation for responses scoring below 60%.

Table 9: Structured description of the Take Mock Interview Session use case

Use Case Attribute	Details
Use Case Name	Take a Mock Interview Session
Primary Actors	User
Description	Users initiate a mock technical interview session. The AI generates domain-specific questions, evaluates responses,

	issues adaptive follow-ups for low scores, and generates a performance feedback report at session completion.
Trigger	User selects a domain and difficulty and starts a new interview session.
Pre-Conditions	User must be authenticated. Domain and difficulty must be selected before session initiation.

3.5.10 View Performance Feedback Report

The "View Performance Feedback Report" use case allows authenticated users to review the detailed performance report generated upon mock interview session completion. This use case is triggered whenever a user navigates to the Reports section of the Mock Interview module. As outlined in Table 3.10, the precondition requires that at least one completed interview session exists in the Session Repository. The normal flow involves the retrieval and display of the selected session report, including per-question scores, strengths, weaknesses, and overall performance band.

Table 3.10: Structured description of the View Performance Feedback Report use case

Use Case Attribute	Details
Use Case Name	View Performance Feedback Report
Primary Actors	User
Description	Users view the detailed performance feedback report generated after a completed mock interview session, including per-question scores, strengths, weaknesses, and overall performance band.
Trigger	User navigates to the Reports section of the Mock Interview module.
Pre-Conditions	At least one completed interview session must exist in the Session Repository.

Post-Conditions	The feedback report is displayed. The user can navigate previous session reports through the session history panel.
Normal Flow	1. Login → 2. Navigate to Mock Interview → 3. Select Reports → 4. System retrieves report from Session Repository → 5. Report displayed with scores, strengths, and weaknesses
Alternative Flow	No completed sessions return "No interview reports available. Complete a session to view your feedback."
Exceptions	Data retrieval error, server error, no internet connection
Business Rules	Only completed sessions generate feedback reports. Session history must be accessible from the user dashboard at all times.

3.5.11 Receive Notifications and Email Alerts

The "Receive Notifications and Email Alerts" use case enables the platform to deliver on-site and email notifications to authenticated users for events, including job match updates, auto-apply submissions, forum activity, and weekly career digest emails. This use case is triggered whenever a notification-generating event occurs within the platform. As outlined in Table 3.11, business rules enforce a maximum of 10 on-site notifications per hour per category and require weekly digest emails to be dispatched every Sunday at 8 AM via SMTP.

Table 10: Structured description of the Receive Notifications and Email Alerts use case

Use Case Attribute	Details
Use Case Name	Receive Notifications and Email Alerts
Primary Actors	User
Description	Users receive on-site and email notifications for platform events. Weekly career digest emails are dispatched every Sunday at 8 AM via SMTP.
Trigger	A notification-generating event occurs within the platform.

Pre-Conditions	User must be authenticated. User email must be verified. SMTP configuration must be active.
Post-Conditions	On-site notification displayed in Notification Center. Email alert dispatched via SMTP. All notifications are logged with a read status and expiry timestamp.
Normal Flow	1. Event occurs → 2. NotificationManager creates an on-site record → 3. Notification displayed in Notification Center → 4. Email dispatched if event qualifies → 5. Weekly digest dispatched every Sunday at 8 AM
Alternative Flow	SMTP failure logs are dispatched as failed and trigger Celery retry.

3.5.12 Admin -Manage User Accounts

The "Admin Manage User Accounts" use case enables the administrator to view, suspend, delete, and assign roles to registered user accounts from the Admin Panel. This use case is triggered whenever the administrator monitors user activity or responds to a reported account issue. As outlined in Table 3.12, the precondition requires that the user account exists in the system. All administrative actions are recorded in the AuditLogs table with the administrator's identity, action type, target entity, and timestamp.

Table 11: Structured description of the Admin Manage User Accounts use case

Use Case Attribute	Details
Use Case Name	Admin Manage User Accounts
Primary Actors	Administrator
Description	Administrators view, suspend, delete, and assign roles to registered user accounts from the Admin Panel.
Trigger	An administrator monitors user activity or responds to a reported account issue.

Pre-Conditions	The administrator must be authenticated. The target user account must exist in the system.
Post-Conditions	Selected account action is executed. The action is recorded in the AuditLogs table with a complete audit trail.
Normal Flow	1. Admin login → 2. Navigate to User Management → 3. Search for user → 4. Select and apply account action → 5. System executes action → 6. Action logged in AuditLogs
Alternative Flow	If the selected account has active pending tasks, the system prompts the administrator for confirmation before deletion.
Exceptions	Server error, database error, or no internet connection.
Business Rules	Only verified administrators can modify user accounts. All actions must be recorded in the AuditLogs table regardless of success or failure.

3.5.13 Admin - Moderate Forum Content

The "Admin Moderate Forum Content" use case allows the administrator to review flagged posts and replies from the moderation queue and take appropriate action, including content removal and user warning issuance. This use case is triggered whenever forum content is flagged by users or the automated moderation filter. As outlined in Table 3.13, business rules enforce that users accumulating three warnings within 30 days are automatically flagged for administrator review.

Table 12: Structured description of the Admin Moderate Forum Content use case

Use Case Attribute	Details
Use Case Name	Admin Moderate Forum Content
Primary Actors	Administrator
Description	Administrators review flagged forum posts and replies, removing violating content and issuing warnings to users who breach platform guidelines.

Trigger	Forum content is flagged by users or the automated moderation filter.
Pre-Conditions	The administrator must be authenticated. Flagged content must exist in the moderation queue.
Post-Conditions	Violating content is removed or approved. A warning is issued to the content author if applicable. Action is logged in AuditLogs.
Normal Flow	1. Admin login → 2. Navigate to Forum Moderation → 3. Review flagged content → 4. Apply removal or approval action → 5. Issue warning if applicable → 6. Action logged
Alternative Flow	Content that is cleared of violation is approved and removed from the moderation queue.
Exceptions	Server error, database error, or no internet connection.
Business Rules	Users accumulating 3 warnings within 30 days are automatically flagged for admin review. All moderation actions must be logged in the AuditLogs table.

3.5.14 Admin Monitor Job Scraper

The "Admin Monitor Job Scraper" use case enables the administrator to review the health and execution status of the scheduled job scraping tasks from the Admin Panel monitoring dashboard. This use case is triggered whenever the administrator checks scraper status or an automated scraper failure alert is generated. As outlined in Table 3.14, the precondition requires that scraper execution logs are available in the system.

Table 13: Structured description of the Admin Monitor Job Scraper use case

Use Case Attribute	Details
Use Case Name	Admin Monitor Job Scraper
Primary Actors	Administrator

Description	Administrators monitor job scraper health, review execution logs, check last-run timestamps, and investigate scraper errors from the Admin Panel monitoring dashboard.
Trigger	Administrator checks scraper status, or a scraper failure alert is generated.
Pre-Conditions	The administrator must be authenticated. Scraper execution logs must be available.
Post-Conditions	Scraper status is reviewed. Errors are identified and flagged for resolution.
Normal Flow	1. Admin login → 2. Navigate to Scraper Monitoring → 3. Review execution status and last-run timestamp → 4. Investigate error logs if failures are present
Alternative Flow	Scraper failure triggers automated admin alert notification.
Exceptions	Log retrieval error, server error, or no internet connection.
Business Rules	Scraper execution logs must be retained for administrator review. Failures must trigger automated alert notifications to the administrator.

3.5.15 Admin View Analytics and Export Audit Logs

The "Admin View Analytics and Export Audit Logs" use case allows the administrator to access platform-wide analytics and export the complete audit log in CSV format for compliance and reporting purposes. This use case is triggered whenever the administrator navigates to the Analytics and Audit Logs section of the Admin Panel. As outlined in Table 3.15, business rules require that audit logs be retained for a minimum of 12 months and that export functionality is restricted exclusively to verified system administrators.

Table 14: Structured description of the Admin View Analytics and Export Audit Logs

Use Case Attribute	Details
Use Case Name	Admin View Analytics and Export Audit Logs
Primary Actors	Administrator
Description	Administrators access platform-wide analytics covering user growth and module engagement, and export the complete audit log in CSV format for compliance and reporting.
Trigger	Administrator navigates to the Analytics and Audit Logs section of the Admin Panel.
Pre-Conditions	The administrator must be authenticated. Audit log records must exist in the AuditLogs table.
Post-Conditions	The analytics dashboard is displayed. Audit log is exported as a CSV file upon request.
Normal Flow	1. Admin login → 2. Navigate to Analytics and Audit Logs → 3. View platform analytics → 4. Select export → 5. System generates and downloads CSV audit log
Alternative Flow	An empty audit log returns "No audit records available for the selected period."
Exceptions	Export generation error, server error, no internet connection
Business Rules	Audit logs must be retained for a minimum of 12 months. Export functionality is restricted exclusively to verified system administrators.

3.6 Functional Requirements

Functional requirements define the specific behaviors and operations that the AI-Powered Career Assistant Platform must provide to satisfy the needs of its users. These requirements outline the essential capabilities that the system must possess to manage user career data, automate job

discovery and application submission, deliver AI-powered career guidance and interview preparation, and support administrative governance effectively. The platform is designed to offer a secure, reliable, and intelligent environment that ensures data integrity, user privacy, and efficient career support service delivery across all ten platform modules.

3.6.1 User Authentication and Profile Management

The platform allows users to register accounts using valid email addresses and secure passwords, with all credentials validated during submission and passwords stored as hashed values through Django's built-in authentication framework. Login is secured through Django's session-based authentication, and accounts are protected through Django's built-in brute force protection mechanisms. Users access their personalized Dashboard upon successful authentication, where all ten platform modules are accessible through a consistent navigation structure. After registration, users can manage and update their professional profiles, including display name, contact details, declared skills, target job role preferences, and Auto-Apply match threshold settings. All profile changes are validated before persistence, and updated skill data is immediately applied to subsequent job match scoring and CV gap detection operations.

3.6.2 CV Analysis and Skill Gap Detection

The system accepts CV uploads in PDF and DOCX formats up to a maximum of 5MB per file. Uploaded files are validated for format and size before being stored in the CV Repository and dispatched as asynchronous Celery tasks to the CV Analyzer. The NLP pipeline extracts skills and qualifications using spaCy's `en_core_web_trf` named entity recognition model, normalizes extracted tokens against a curated skill taxonomy, and computes a four-dimensional quality score weighted across keyword relevance, completeness, skill density, and formatting quality.

Skill gaps are identified by comparing extracted skills against the top 50 in-demand skills for the user's target role, with each detected gap mapped to a corresponding Resource Hub learning recommendation. All analysis results are persisted to the Analysis Results store and returned to the user dashboard upon Celery task completion.

3.6.3 Job Discovery and Match Scoring

The system scrapes job listings from LinkedIn, Indeed, and Rozee.pk through Django management commands and view-triggered scraping workflows, retrieving structured job data using BeautifulSoup4 and Selenium, and persisting listings to the database. Each listing undergoes match score computation using cosine similarity between the job description and the user's skill profile, computed through lightweight sentence embeddings. Users can search and filter listings by job role, location, and work modality, receiving a paginated feed ranked by match percentage. Listings older than 30 days are automatically purged from the database through scheduled Django management commands, and scrapers apply a minimum 2-second throttle between portal requests to comply with rate-limiting constraints.

3.6.4 Auto-Apply Automation

When a user enables the Auto-Apply toggle, the Auto-Apply module queries the database for all listings meeting the user's defined match threshold and checks each against the application history to enforce the 30-day duplicate prevention window. Eligible listings are processed through Selenium-based form automation triggered via Django views, with each submission attempt handled sequentially. All submission attempts are logged in the Applications table with job reference, submission timestamp, status, and failure reason where applicable. On-site notifications are dispatched upon each successful or failed submission, ensuring the user maintains full visibility of their automated application activity.

3.6.5 AI Chatbot and Career Guidance

The platform provides a conversational AI chatbot interface accessible as a persistent side panel across all platform screens. User career queries are processed by the AI chatbot module, which generates contextually relevant responses based on the user's query content and active session conversation history using spaCy-based text processing and predefined response logic.

The chatbot delivers responses within a maximum of 3 seconds of message submission and maintains full conversation context within the active session. Chat history is stored in the Chat History data store for reference within the current session. Job recommendations surfaced by the chatbot are drawn from the Job Repository and filtered by the user's skill profile match score.

3.6.6 Resource Hub and Learning Recommendations

The Resource Hub displays curated learning resources from Coursera and Udemy, categorized by skill, proficiency level, and cost. Personalized recommendations are automatically generated from the skill gap results produced during the user's most recent CV analysis, linking each detected gap to a set of targeted learning resources. Users can also manually search the Resource Hub and apply filters to browse the full resource library independently of their CV analysis results. Resources are updated periodically to reflect current course availability from integrated external learning platforms.

3.6.7 Community Forum and Gamification

The Community Forum allows authenticated users to create posts, submit replies, and like content within categorized discussion sections. The gamification engine awards 10 contribution points per new post, 5 points per reply, and 2 points per received like, evaluating badge tier eligibility after each point-awarding event. Badge tiers are assigned at fixed thresholds: Newcomer at registration, Contributor at 50 points, Expert at 200 points, and Champion at 500 points. The leaderboard ranks all users by total contribution points in descending order and is refreshed after each point-awarding event. Flagged content is routed to the admin moderation queue, and users accumulating three warnings within 30 days are automatically flagged for administrator review.

3.6.8 Mock Interview Simulation

The Mock Interview Module enables users to initiate adaptive technical interview sessions by selecting a domain from Database, OOP, and Mixed options, and a difficulty level from Beginner, Intermediate, and Advanced tiers. The MockInterviewModule generates domain-specific questions using the TransformerModel, evaluates each submitted response for conceptual accuracy, technical depth, and clarity using semantic similarity scoring, and assigns a normalized score mapped to one of four performance bands: Poor (0–40), Average (41–60), Good (61–80), and Excellent (81–100). Responses scoring below 60% trigger adaptive follow-up question generation targeting the specific conceptual weakness identified. Upon session completion, a comprehensive feedback report is generated within 10 seconds, detailing per-question scores, identified strengths and weaknesses, and overall performance band. The report is persisted to the Session Repository and displayed on the user dashboard, with full session history accessible at all

times.

3.6.9 Notification System

The Notification Module delivers on-site alerts for all platform events, including job match updates, auto-apply submission confirmations, forum activity, and system announcements. On-site notifications are displayed in the Notification Center and grouped to prevent notification fatigue. Email notifications are dispatched via SMTP using Django's built-in email backend. A weekly career digest email summarizing job matches, application activity, and resource recommendations is dispatched through a scheduled Django management command. All notification records are retained in the Notifications table for 90 days with read status and expiry timestamps.

3.6.10 Admin Panel and Governance

The Admin Panel provides the administrator with comprehensive platform governance tools accessible through a role-restricted, tabbed management interface. User account management enables viewing, suspending, deleting, and role-assigning registered accounts. Forum moderation provides access to the flagged content queue with removal, approval, and warning issuance controls. Scraper monitoring displays last-run timestamps, execution status, and error logs for all scheduled scraping tasks. Auto-apply task oversight provides visibility into queued, active, and completed submission tasks. Platform analytics present user growth trends and module engagement metrics. Audit log management enables viewing and CSV export of all administrative actions, retained for a minimum of 12 months.

3.7 Non-Functional Requirements

Non-functional requirements define the quality standards, performance thresholds, and operational constraints that the AI-Powered Career Assistant Platform must satisfy beyond its core functional capabilities. These requirements govern how well the system performs its functions rather than what functions it performs, ensuring that the platform delivers a reliable, secure, accessible, and maintainable career support experience for all users.

3.7.1 Performance

The system must deliver consistently responsive performance across all user-facing operations. The main Dashboard must load within 3 seconds on a standard broadband connection. CV analysis

must be completed within 30 seconds of processing initiation.

Database queries with applied filters must return results within 2 seconds with proper indexing applied through Django's ORM. AI chatbot responses must be delivered within 3 seconds of message submission. Mock interview feedback reports must be generated and displayed within 10 seconds of session completion. The system must support a minimum of 500 concurrent authenticated users without performance degradation, and background processing tasks, including job scraping and auto-apply submissions, must be handled without blocking core user-facing views.

3.7.2 Scalability

The system must be designed to accommodate growth in registered users, job listings, forum content, and interview session data over time. The SQLite database schema must support a minimum of 100,000 user records, 1,000,000 job listings, and 500,000 forum posts without requiring structural modifications. The job scraper architecture must be designed to support the addition of new portal integrations without changes to the core scraping infrastructure. Background processing tasks must support scaling through the addition of Django management commands and view-level optimizations to handle increased task volumes as the user base grows.

3.7.3 Security

Security is a critical non-functional requirement given the sensitive nature of user career data and professional profile information stored within the platform. User passwords must be stored exclusively as hashed salted values through Django's built-in authentication framework. Role-based access control must restrict all administrative endpoints to verified administrators through Django's built-in permissions system, returning HTTP 403 for unauthorized access attempts. All administrative actions must be recorded in an audit log retained for a minimum of 12 months. Cross-user data isolation must be enforced at the Django ORM query level, preventing any authenticated user from accessing another user's CV data, application history, or interview session records. Django's built-in CSRF middleware must be enforced across all form submissions and state-modifying requests.

3.7.4 Reliability and Availability

The platform must maintain a minimum monthly uptime across all user-facing services. Failed background tasks must be logged with failure reasons and retry mechanisms implemented through Django view-level error handling. Module-level failures must not propagate to affect the availability of other platform modules, ensuring graceful degradation under partial system failure conditions. The system must implement atomic database transactions for all multi-step operations, including auto-apply submission and forum gamification point awards, preventing partial state corruption.

3.7.5 Usability

The platform interface must be intuitive and accessible for users with varying levels of technical expertise. A new user must be able to complete the full onboarding flow, including registration, profile setup, and CV upload, within 10 minutes without external guidance. The interface must conform to WCAG 2.1 Level AA accessibility standards, including ARIA labels, a minimum contrast ratio of 4.5:1, and full keyboard navigation support across all screens. All error messages must clearly state the issue and provide a corrective action to guide the user toward resolution. The interface must be responsive and optimized for viewport widths between 1280px and 1920px.

3.7.6 Maintainability

The platform must be designed for ease of maintenance, extension, and long-term evolution. All backend views, models, and utility functions must include inline docstrings describing their purpose, parameters, and return values. The codebase must follow the Model-View-Template architectural pattern throughout the Django backend. All source code must be maintained in a Git repository for version control and collaborative development. Structured application logs must be maintained with a 30-day rotation policy. The AI model integration must be implemented through an abstraction layer allowing model upgrades without changes to the consuming business logic classes.

3.7.7 Compliance

The system must handle all user data in accordance with applicable data protection principles, ensuring that personal information, including email addresses, CV content, skill profiles, and application history, is stored securely, accessed only by authorized parties, and retained only for

defined periods aligned with the data retention policies specified in the system's non-functional requirements.

CV files must be stored with restricted access controls. Application history records must be retained for a minimum of 12 months. Notification logs must be purged after 90 days. All data purge operations must be executed automatically through scheduled Django management commands.

3.7.8 Backup and Recovery

The platform must implement regular automated backups of the SQLite database to ensure that user profiles, CV analysis results, job listings, application history, forum content, interview session data, and audit logs can be restored in the event of system failure or data corruption. Recovery procedures must be tested periodically to verify effectiveness and minimize potential data loss and service downtime in emergency scenarios. Critical background processing workflows must be configured with error handling and logging to minimize disruption in the event of unexpected process termination.

3.7.9 Performance Monitoring and Logging

The system must include mechanisms for monitoring performance, logging user activity, and detecting operational anomalies. Structured application logs must capture all view request events, background task execution outcomes, authentication events, and error conditions through Django's built-in logging framework. Django's admin panel must provide real-time visibility into platform activity, user management, and data inspection for administrator monitoring. Log rotation must be configured to retain application logs for a minimum of 30 days before automated purging.

Chapter 4

DESIGN AND ARCHITECTURE

4. Design and Architecture

The Design and Architecture chapter of the AI-Powered Career Assistant Platform explains how the software is structured, how its components interact, and how data flows throughout the system. This chapter provides a detailed overview of the system's architecture, data representation, process flows, and design models. It highlights the technical decisions made to ensure performance, security, scalability, and maintainability across all ten platform modules. By presenting the system design clearly through architectural diagrams, class models, sequence diagrams, state transition diagrams, and data flow diagrams, this chapter demonstrates how the AI-Powered Career Assistant Platform meets both functional and non-functional requirements while delivering a reliable, intelligent, and accessible career support ecosystem.

4.1 System Architecture

The system architecture defines the overall structure of the AI-Powered Career Assistant Platform and illustrates how its main components work together to deliver career support services efficiently. It ensures that each module performs its specific tasks while interacting seamlessly with others to maintain smooth operations, data integrity, and consistent system performance across all user interactions.

4.1.1 Architecture Style

The AI-Powered Career Assistant Platform is developed using a layered client-server architecture organized into functional layers, each responsible for a distinct set of system concerns. The Presentation Layer consists of Django-rendered HTML and CSS templates delivering the user interface across all ten platform modules through a responsive, browser-based frontend. The Application Layer is implemented using Django, handling all request routing through Django's URL dispatcher and managing business logic through Django's view layer.

The separation of concerns enforced by this layered architecture enhances maintainability, supports scalability as the user base grows, and ensures secure and organized handling of all system operations from user interface interaction through to data persistence and AI model invocation.

4.1.2 Components

The AI-Powered Career Assistant Platform is composed of several interconnected components, each designed to perform a specific function within the overall architecture. These components

work collaboratively to ensure smooth system operation, secure data handling, and efficient career support service delivery. By dividing the system into modular parts, the platform maintains clarity in functionality, supports easier maintenance, and allows individual components to evolve or be upgraded without affecting the entire system.

- **User Interface:** Implemented using Django's built-in templating engine with HTML and CSS styling, providing a consistent, responsive, and accessible interface across all ten modules. The interface uses the Inter typeface and consistent iconography, conforms to WCAG 2.1 Level AA standards (ARIA labels, 4.5:1 contrast ratio, full keyboard navigation), and communicates with the backend via standard HTTP requests and Django form submissions.
- **Authentication and Security Service:** Manages user registration, login, session management, and role-based access control across protected views. It enforces strict separation between administrators and regular users using Django's permissions and middleware system. Security measures include password hashing, session-based authentication, account protection after consecutive failed login attempts, and CSRF protection via Django's middleware.
- **CV Analyzer Module:** Accepts PDF and DOCX uploads, validates format and size constraints, and processes files through the NLP pipeline within the Django view layer. It uses spaCy for named entity recognition and skill extraction, and lightweight sentence embeddings for semantic scoring and gap detection. Results (quality score, extracted skills, gaps, and improvement suggestions) are stored in the database and displayed on the user dashboard.
- **Job Scraper and Match Scoring Module:** Executes on-demand and scheduled scraping via Django management commands and view-triggered workflows using BeautifulSoup4 and Selenium to retrieve listings from LinkedIn, Indeed, and Rozee.pk. Each listing undergoes cosine similarity-based match scoring using lightweight sentence embeddings. Listings are stored with computed match scores and automatically purged after 30 days via scheduled commands.
- **Auto-Apply Manager:** Automates job applications by selecting listings that meet the user-defined match threshold, enforcing a 30-day duplicate prevention policy through application history checks, and submitting forms via Selenium automation triggered by Django views. All attempts are logged with status, timestamp, and failure reason, and notification events are sent

to the Notification Module.

- **AI Chatbot Service:** Provides real-time career guidance via WebSocket connections managed by Django Channels 4.x. User queries are processed by the AIChatbot class, which invokes the TransformerModel to generate context-aware responses while maintaining session context. Chat history is stored in the Chat History data store, and job recommendations are retrieved from the Job Repository based on the user's skill profile.
- **Mock Interview Module:** Conducts adaptive technical interview sessions in Database, OOP, and Mixed domains at three difficulty levels. It generates questions using the TransformerModel, evaluates answers through semantic similarity scoring, issues adaptive follow-up questions for responses below 60%, and produces comprehensive performance reports. All session data and feedback are stored in the Session Repository.
- **Community Forum and Gamification Engine:** Enables categorized posts and reply threading with like support. The gamification engine awards contribution points, evaluates badge eligibility after each event, and maintains a real-time leaderboard. Flagged content is automatically routed to the admin moderation queue.
- **Resource Hub:** Provides curated learning recommendations from Coursera and Udemy based on CV skill gap analysis, categorized by skill, proficiency level, and cost. Users can also manually search and filter the full resource library independently of CV analysis results.
- **Notification Service:** Delivers on-site alerts and email notifications through a unified pipeline. On-site notifications are grouped in the Notification Center to reduce fatigue, while email alerts and weekly career digests are sent via SMTP using Django's email backend. Weekly digests are triggered through scheduled Django management commands.
- **Admin Panel:** Offers a role-restricted tabbed interface for user management, forum moderation, scraper monitoring, auto-apply oversight, analytics, and CSV audit log export. All administrative actions are recorded in the AuditLogs table and retained for a minimum of 12 months.
- **Task Queue Infrastructure:** Handles long-running operations such as job scraping, CV analysis, auto-apply submission, and email digest generation using Django management

commands and view-level processing. Failed tasks are logged with timestamps and failure reasons for administrator review. Django's logging framework provides visibility into execution outcomes and error conditions.

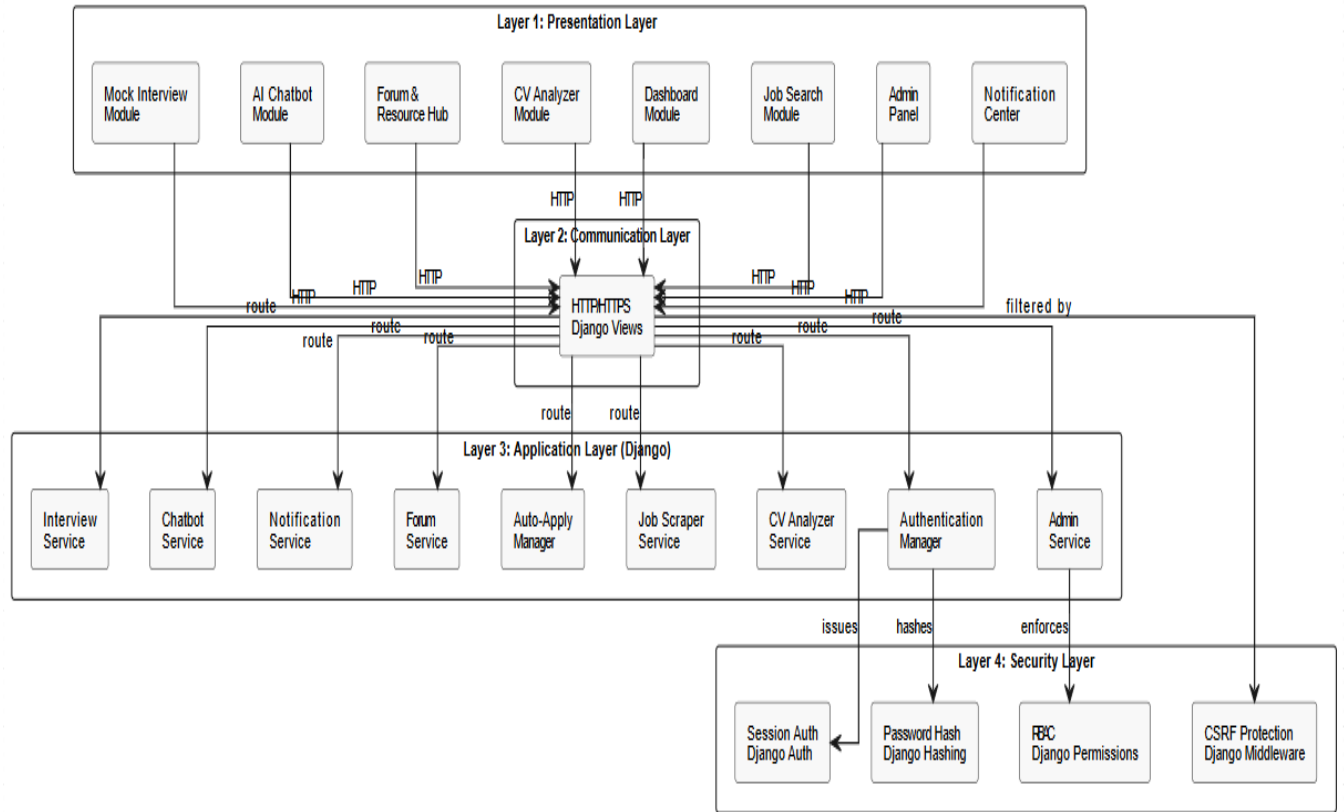


Figure 6: System Architecture - Presentation & Application Layers

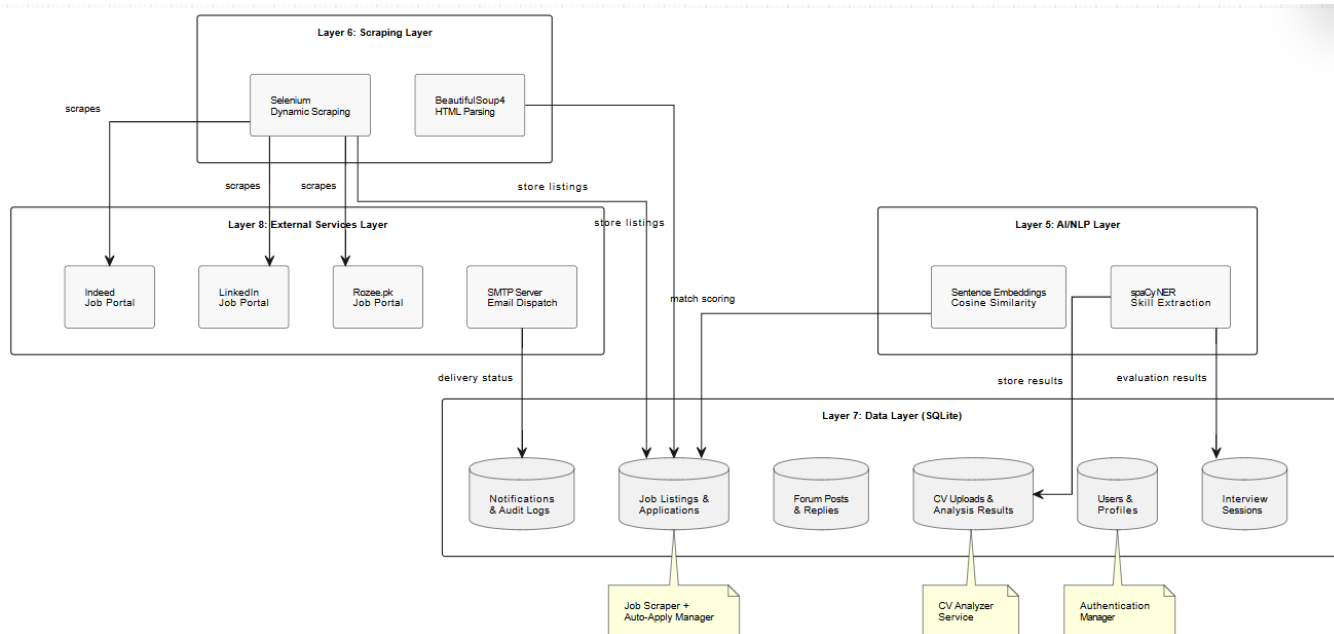


Figure 7: System Architecture - AI/ML, Task Queue, Data & External Layers

4.2 Data Representation

The data representation of the AI-Powered Career Assistant Platform defines how all system information is structured, stored, and related within the SQLite relational database. This section presents the database design, entity-relationship representation, data flow across system layers, and entity descriptions and constraints governing all primary data entities within the platform.

4.2.1 Database Design

The platform's data model is implemented as a normalized SQLite relational schema managed through Django's ORM, comprising ten primary entities. The Users table stores authentication credentials, role assignments, and account status flags. The User Profiles table extends Users with professional information, including declared skills, job preferences, match threshold settings, and Auto-Apply toggle state. The CV Uploads table records file metadata, extracted skill vectors, quality scores, and gap analysis results associated with each uploaded CV.

The JobListings table persists scraped job data, including title, description, portal source, location, work modality, expiry date, and computed match scores. The Applications table logs every auto-apply submission attempt with job reference, submission timestamp, status, and retry count. The

ForumPosts and ForumReplies tables store community content with author references, category tags, like counts, and moderation flags. The Notifications table records on-site alerts with type, read status, and expiry timestamp. The InterviewSessions table captures mock interview metadata, including domain, difficulty, question sequence, response scores, and overall performance band. The AuditLogs table records all administrative actions with actor identity, action type, target entity, and timestamp, retained for a minimum of 12 months.

4.2.2 Entity-Relationship Representation

The entity-relationship model of the platform defines the associations between all primary data entities and the constraints governing those associations. The Users entity maintains a one-to-one relationship with the UserProfiles entity, as each registered user has exactly one professional profile record. Users maintain a one-to-many relationship with CVUploads, as each user may upload multiple CV versions over time. Users maintain a one-to-many relationship with Applications, as each user may submit multiple applications over time. Users maintain a one-to-many relationship with InterviewSessions, as each user may take multiple interview sessions over time. UserProfiles maintain a one-to-many relationship with JobListings, as each user profile may match multiple job listings. Applications maintain a one-to-many relationship with JobListings, as each application may reference multiple job listings.

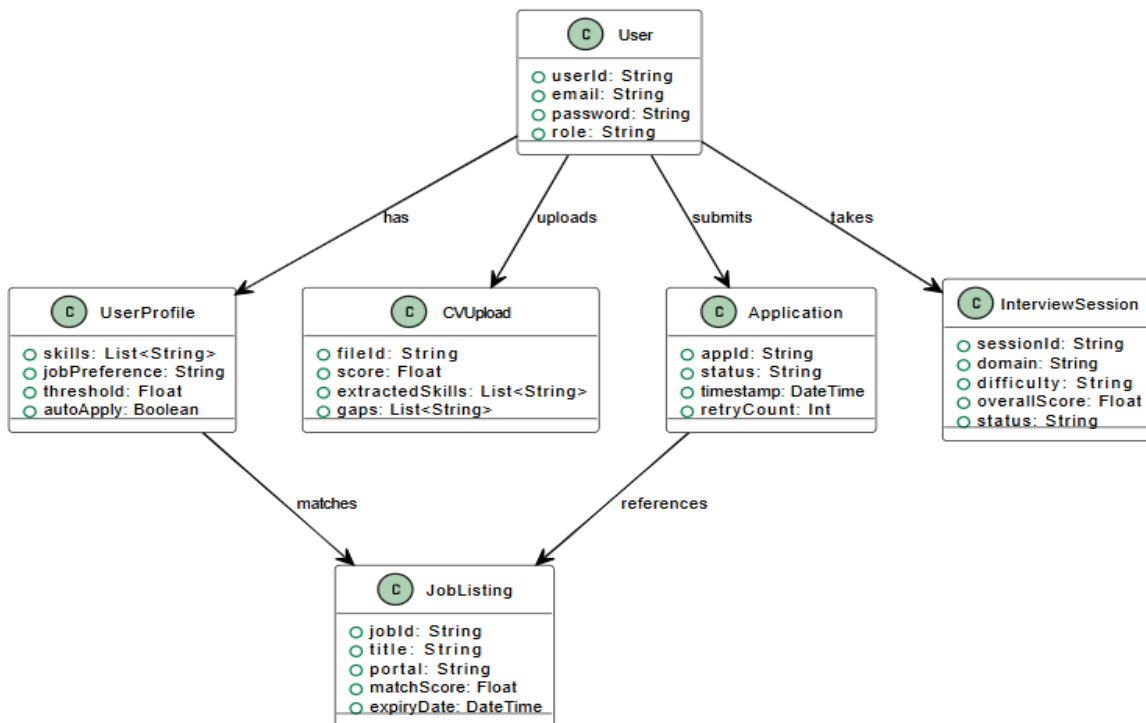


Figure 8: Class Diagram Core Classes

The UserProfiles entity maintains a one-to-many relationship with Applications, as each user may accumulate multiple application submission records. JobListings maintains a one-to-many relationship with Applications, as each listing may be applied to by multiple users across the platform. Users maintain a one-to-many relationship with ForumPosts and ForumReplies, reflecting each user's contribution history within the Community Forum. Users maintain a one-to-many relationship with InterviewSessions, as each user may complete multiple mock interview sessions across different domains and difficulty levels.

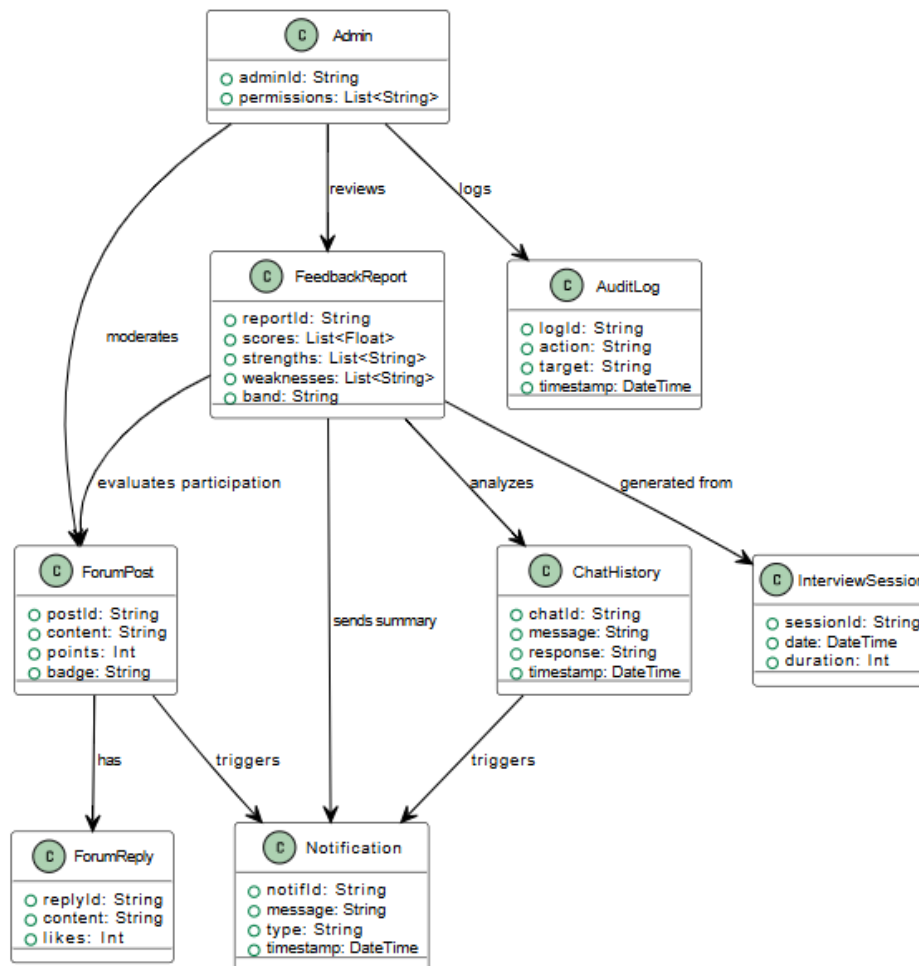


Figure 9: Class Diagram - AI/ML & Feature Classes

4.2.3 Data Flow

The platform's data flows are organized across three major operational pipelines, each capturing the movement of information between external entities, internal processing components, and persistent data stores.

- **Sequence Diagram (Data Flow Across Layers)**

The sequence diagrams capture the temporal ordering of interactions between system components across the platform's three primary processing pipelines. The Authentication and CV Analysis sequence illustrates how user credentials flow from the React.js frontend through the Django API to the AuthenticationManager, which validates identity against the UserRepository and returns a JWT token, before the user initiates a CV upload that triggers an asynchronous Celery task dispatching through the CVAnalyzer to the NLP Engine and returning scored results to the frontend. The Job Scraping and Auto-Apply sequence illustrates the scheduled scraping cycle initiated by the TaskQueue, the match scoring pipeline applied to each retrieved listing, and the auto-apply submission workflow triggered by user toggle activation. The AI Chatbot and Mock Interview sequence illustrates real-time WebSocket message processing through the AIChatbot and MockInterviewModule, including adaptive follow-up question generation and session report persistence.

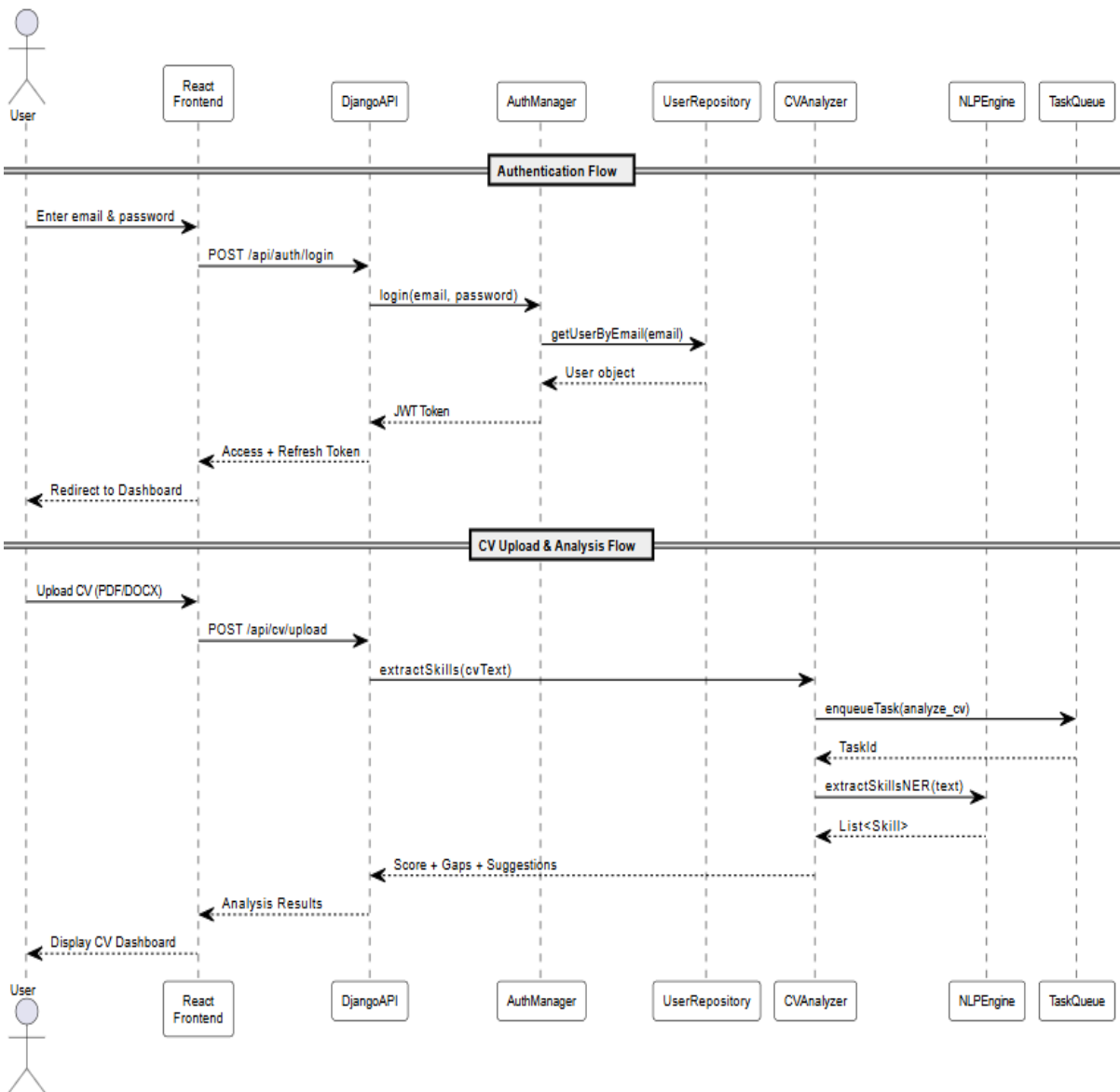


Figure 10: User Authentication & CV Analysis Flow (Sequence Diagrams)

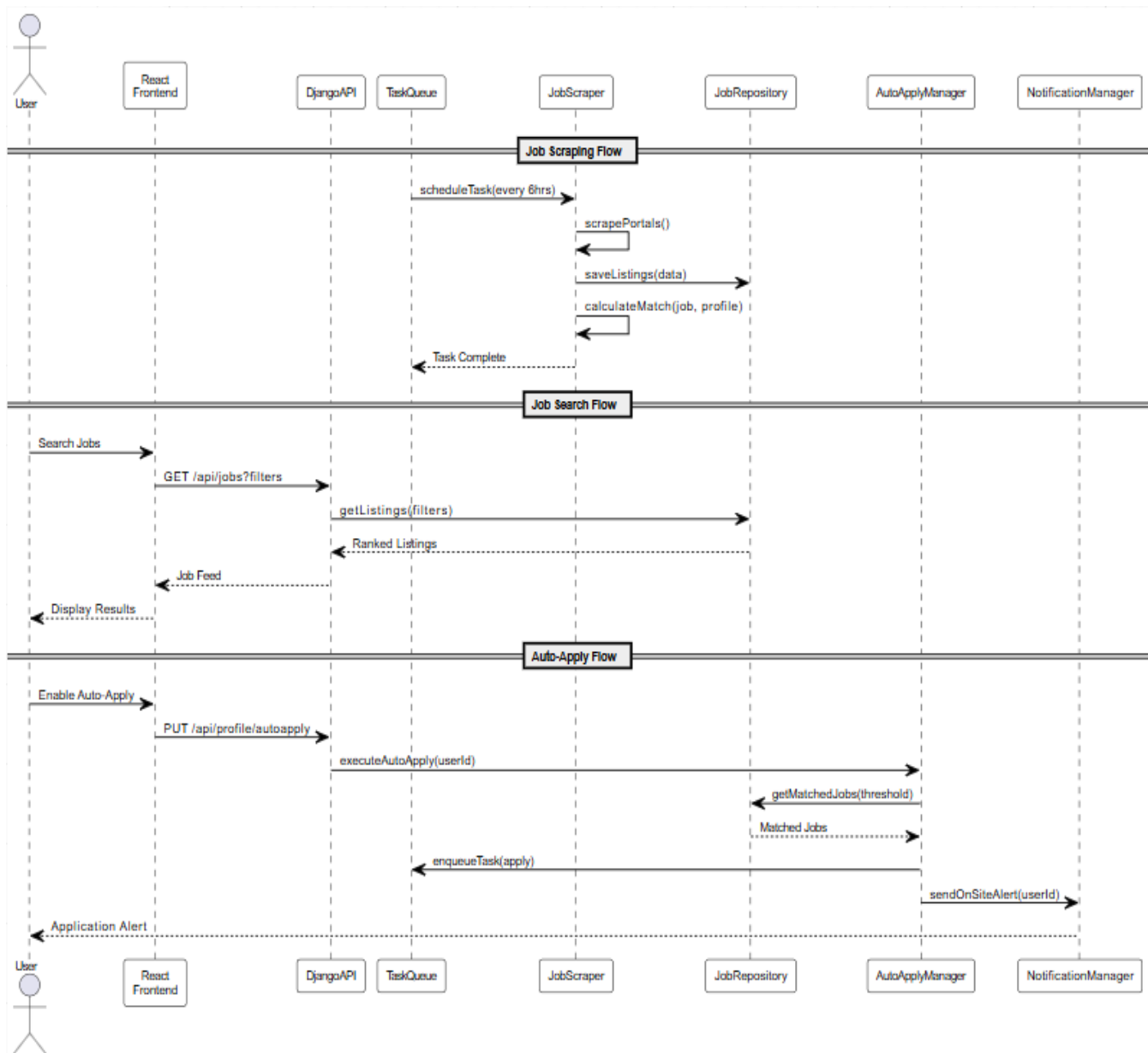


Figure 11: Job Scraping & Auto-Apply Flow (Sequence Diagrams)

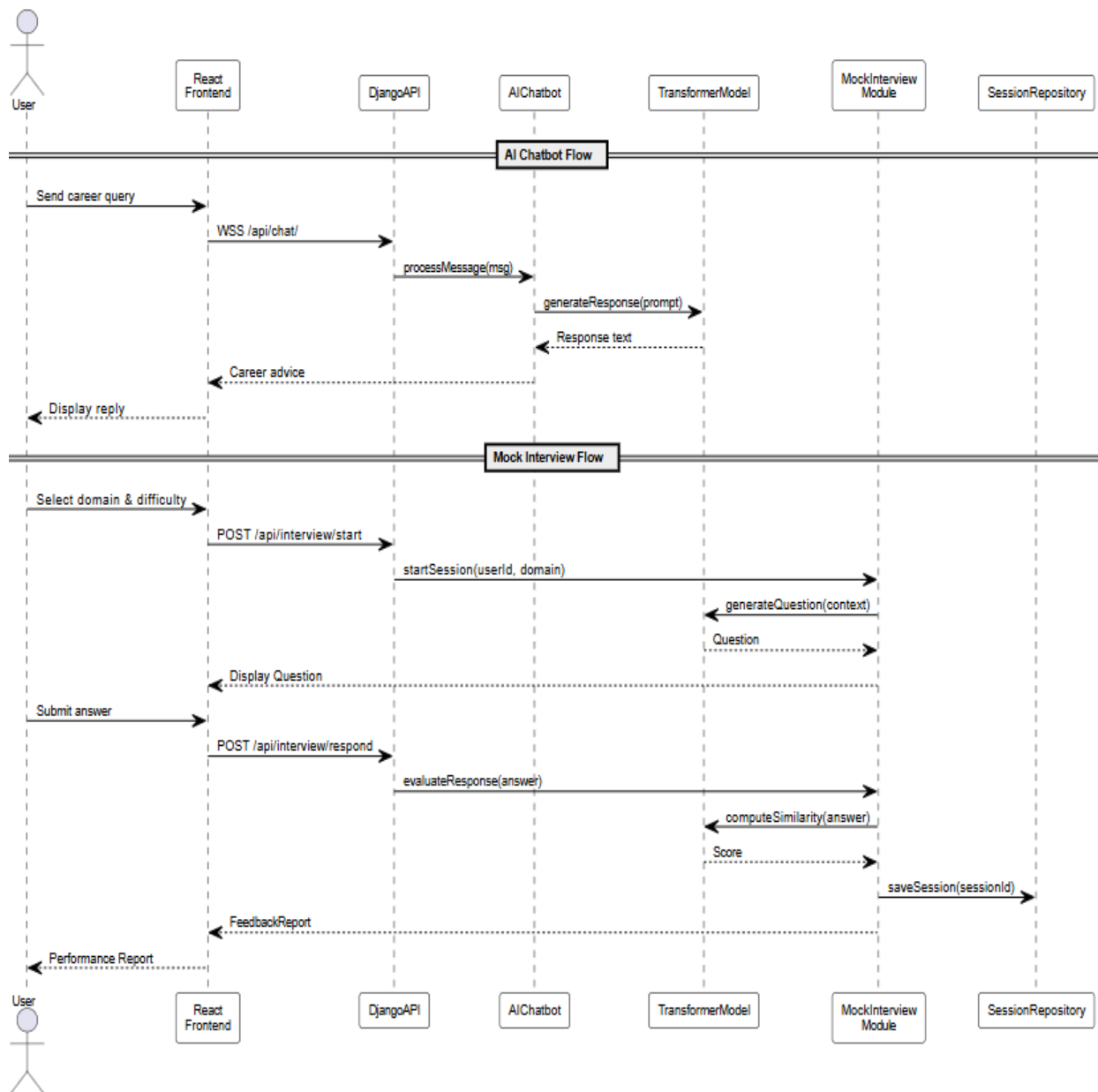


Figure 12: AI Chatbot & Mock Interview Flow (Sequence Diagrams)

- **Activity Diagram (Overall System Workflow)**

The state transition diagram illustrates the complete lifecycle of user interactions within the AI-Powered Career Assistant Platform, beginning from application launch through a central Dashboard hub that governs all module transitions. The system initializes in an Idle state, transitions through Authentication upon credential verification, and enters the Dashboard from which all module states are reachable. Each module implements its own internal state lifecycle, and all module states terminate back to the Dashboard upon completion, ensuring a consistent and navigable user experience throughout the platform lifecycle.

The workflow continues with the Resume Intelligence module, where the uploaded CV is processed through an NLP analysis pipeline to extract relevant competencies, qualifications, and experience indicators. Based on this extracted information, the system identifies skill gaps by comparing the user's profile with current job market requirements obtained through automated job listing scraping. These listings are then ranked according to relevance, enabling the user to perform refined searches and apply filtering criteria before activating the auto-application feature. Once enabled, the system automatically submits applications to matching opportunities, significantly reducing manual effort and improving efficiency in the job application process.

Following the application phase, the user interacts with the AI Chatbot module to receive personalized career guidance and resource hub recommendations tailored to identified improvement areas. The workflow then transitions into the Mock Interview module, where users participate in simulated interview sessions designed to evaluate communication skills and technical readiness. Upon completion, the system generates a structured performance report summarizing strengths, weaknesses, and improvement suggestions, which is presented through the Dashboard interface. This ensures that users receive continuous feedback and maintain visibility over their progress within a unified and integrated platform environment.

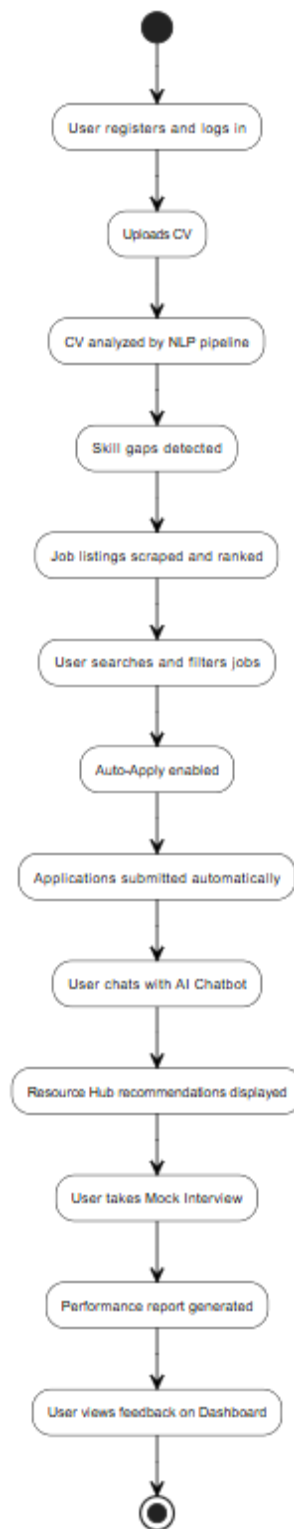


Figure 13: Activity Diagram - User Journey

4.2.4 Entity Descriptions and Constraints

Each primary entity within the platform's data model is governed by specific field-level constraints ensuring data integrity and consistency. The Users entity enforces a unique constraint on the email field, a non-null constraint on password hash and role fields, and a default value of active for the account status field. The UserProfiles entity enforces referential integrity through a foreign key constraint on the user identifier and applies default values of null to optional skill and preference fields until populated by the user. The CVUploads entity enforces a maximum file size constraint of 5MB at the application layer and restricts accepted MIME types to PDF and DOCX formats. The JobListings entity enforces a 30-day expiry constraint managed through a scheduled Celery purge task and applies a non-null constraint on source portal, title, and description fields. The Applications entity enforces a composite unique constraint on the user identifier and job listing identifier within a 30-day window to implement the duplicate prevention policy. The InterviewSessions entity enforces a non-null constraint on domain, difficulty, and status fields, and restricts the status field to values of active, completed, and incomplete. The AuditLogs entity enforces non-null constraints on all fields and applies an immutability policy preventing record deletion or modification after initial insertion.

4.3 Process Flow / Representation

The platform's process flows capture how data moves through the system and how components interact during the execution of all major operational workflows. The following subsections describe the four primary process flows of the AI-Powered Career Assistant Platform.

4.3.1 Login and Authentication Process

The login and authentication process is initiated when a user submits credentials through the Django-rendered login page. The form submission dispatches a POST request to the authentication view, where Django's URL dispatcher routes the request to the Authentication Manager. The Authentication Manager queries the Users table to retrieve the user record associated with the submitted email address, verifies the submitted password against the stored hash using Django's built-in authentication framework, and checks the account lockout status.

Upon successful verification, a Django session is created, and the user is redirected to the Dashboard. If credentials are invalid, an error message is returned and displayed on the login page. If consecutive failures are detected, the account is temporarily locked, and a lockout notification is displayed. All authentication events are logged in the Audit Logs table for security monitoring.

4.3.2 CV Upload and Analysis Process

The CV upload and analysis process begins when an authenticated user selects or drags a CV file onto the CV Analyzer upload interface. The frontend dispatches a POST request to `/api/cv/upload/` with the file attached as multipart form data. The Django API validates the file format and size, stores the raw file in the CV Repository, and dispatches an asynchronous Celery task to the CV Analyzer. The CVAnalyzer extracts raw text from the file using PyMuPDF for PDF files or python-docx for DOCX files, passes the text to the NLP Engine for spaCy-based skill extraction, and invokes the TransformerModel to compute semantic quality scores across four weighted dimensions. Skill gap detection compares extracted skills against the curated in-demand skill taxonomy for the user's target role. All results are persisted to the Analysis Results store and returned to the frontend, where they are displayed as a structured analysis dashboard panel.

4.3.3 Job Scraping and Match Scoring Process

The job scraping and match scoring process is initiated through Django management commands and view-triggered scraping workflows executing on a scheduled basis. The Job Scraper service invokes BeautifulSoup4 and Selenium to retrieve job listings from LinkedIn, Indeed, and Rozee.pk, applying a minimum 2-second throttle between requests to comply with portal rate limiting constraints. Retrieved listings are normalized into a standardized structure and stored in the Job Listings table through Django's ORM. Lightweight sentence embeddings then encode each listing's description and the user's skill profile, computing a cosine similarity match score for each listing. Users searching for jobs submit filter criteria through the Django-rendered job search page, which the corresponding Django view routes to the database query layer, returning a paginated set of listings ranked by match score.

A scheduled Django management command automatically purges listings older than 30 days from the database at each execution cycle.

4.3.4 Mock Interview Session Process

The mock interview session process begins when an authenticated user selects a domain and difficulty level and starts a session from the Mock Interview module. The request is handled by the Django backend, where the system processes the input and creates a new interview session record in the database. The system then retrieves a set of predefined domain-specific questions from the stored question bank based on the selected parameters and presents the first question to the user through the rendered interface.

4.4 Design Models

The design models of the AI-Powered Career Assistant Platform are expressed through four categories of UML diagrams, each capturing a distinct dimension of the system's structure and behavior. Together, these models provide a complete technical specification of the platform's object-oriented architecture, dynamic interaction patterns, state lifecycle management, and data flow organization.

4.4.1 Class Model Description

The class diagram presents the complete object-oriented structure of the platform, comprising nineteen classes organized in a top-down hierarchical layout. The Application class serves as the system entry point, delegating responsibilities to the AuthenticationManager and UserProfileManager. Core feature classes, including CVAnalyzer, JobScraper, AutoApplyManager, and NotificationManager, form the middle tier, each dependent on the TaskQueue for asynchronous processing and their respective repository classes for data persistence. The AI and ML layer comprises the NLP Engine powered by spaCy and the TransformerModel powered by Hugging Face MiniLM-L6-v2, consumed by the CVAnalyzer, JobScraper, AIChatbot, and MockInterviewModule for all intelligent processing operations. Repository classes, including UserRepository, JobRepository, and SessionRepository, form the data access layer, abstracting all database interactions from business logic classes and ensuring clean separation of concerns throughout the codebase.

4.4.2 Sequence Model Description

Three sequence diagrams capture the dynamic interaction flows between system components across the platform's major operational pipelines. Figure 4.2.3.1a illustrates the Authentication and

CV Analysis flow, showing the temporal ordering of credential validation, JWT issuance, CV file upload, Celery task dispatch, NLP processing, and result return. Figure 4.2.3.1b depicts the Job Scraping and Auto-Apply pipeline, covering the scheduled scraper execution cycle, match score computation, user-triggered job search filtering, and auto-apply task enqueueing with notification dispatch. Figure 4.2.3.1c captures the AI Chatbot conversation flow, including WebSocket connection establishment and real-time response generation, and the complete Mock Interview session lifecycle from domain selection through adaptive questioning to feedback report generation and session persistence.

4.4.3 Activity Model Description

The state transition diagrams presented as Figures 4.2.3.2 illustrate the complete lifecycle of user interactions within the platform from application launch through authenticated Dashboard access to individual module state flows. Figure 4.6a covers the application launch, authentication, CV Analysis, Job Search, and Auto-Apply state flows, showing the sequential and conditional transitions between system states for each operational pipeline. Figure 4.6b covers the AI Chatbot, Mock Interview, and Community Forum state flows, capturing the cyclic nature of chatbot conversation, the adaptive follow-up logic of the interview evaluation pipeline, and the post-and-reply threading states of the forum interaction model. All module states terminate back to the Dashboard upon completion, and the logout transition from any state returns the system to the Idle state.

Chapter 5

IMPLEMENTATION

5. Implementation

This chapter explains how the Job Pilot - AI Powered Career Assistant and Job Automation Platform was implemented at the technical level. It does not include source code but provides descriptions of the algorithms, external APIs, background task execution, AI processing logic, and user interface decisions that were used to build the system. The goal of this chapter is to show how the system's design was transformed into a functional software solution while maintaining efficiency, accuracy, security, and scalability.

5.1 Algorithms

This section describes the main algorithms used in the Job Pilot platform. Each algorithm represents the internal logic behind core system features and explains how different functions operate. These descriptions are presented in natural language (with step-by-step workflow) to clearly communicate the logic behind major modules such as authentication, CV analysis, job scraping, auto-apply, and AI-driven services.

5.1.1 Authentication Algorithm

The authentication algorithm ensures that only legitimate users can access the Job Pilot platform. It protects sensitive user profiles, CV data, job applications, and administrative controls by verifying credentials and granting role-based access. This algorithm is crucial for maintaining system security, preventing unauthorized access, and supporting JWT-based session management across the web application.

Steps:

1. User enters email and password on the login screen.
2. The Django API forwards the credentials to the AuthenticationManager.
3. The AuthenticationManager retrieves the user record from the UserRepository and validates the email.
4. The entered password is compared with the securely stored (encrypted) password.
5. If credentials match: a. Generate a secure JWT authentication token containing user ID and role (User / Admin). b. Identify the user role and associated permissions. c. Return the JWT

token to the frontend and redirect the user to the personalized dashboard.

6. If credentials do not match: a. Record the failed login attempt in the audit log. b. If failed attempts exceed the allowed limit, temporarily lock the account for security. c. Notify the user of invalid login credentials and suggest password recovery options.

5.1.2 Mock Interview Session Initiation Algorithm

This algorithm allows users to start AI-powered mock interview sessions efficiently while ensuring proper session management and preventing concurrent session conflicts. It ensures that interview parameters (domain and difficulty) are validated, questions are dynamically generated, and notifications are sent to the user, improving overall preparation workflow and interview experience.

Steps:

1. User selects desired domain (job role) and difficulty level for the mock interview.
2. Retrieve user profile and previous interview history for the selected domain.
3. Check if an active interview session already exists for the user.
4. If slot/session is available: a. Confirm session initiation. b. Save session details (domain, difficulty, timestamp) in the Session Repository. c. Notify the user and trigger the first set of role-specific questions via the TransformerModel.
5. If the session is unavailable or in progress: a. Prompt the user to either resume the existing session or select another domain/difficulty.

5.1.3 CV Analysis and Feedback Generation Algorithm

The CV analysis and feedback generation algorithm allows the system to create accurate, AI-driven CV evaluations and improvement recommendations within the platform. It ensures that feedback is only generated after proper CV upload and validation and that the results are properly linked to the user's profile and job-matching records for future reference.

Steps:

1. User uploads CV (PDF/DOCX) through the CV Analyzer interface.

2. Extract text and structured information using the NLP Engine (spaCy + Hugging Face Transformers).
3. Verify that all required fields (skills, experience, education) are parsed correctly.
4. Check user authorization and CV file validity.
5. If authorized and data is valid: a. Save extracted CV data and analysis results in the CV Repository. b. Link the analysis to the user profile and generate compatibility scores with job listings.
6. Confirm the user and make the detailed feedback report (skill gaps, strengths, suggestions) accessible on the dashboard for future reference.

5.1.4 Auto-Apply Execution Algorithm

This algorithm manages the secure and automated submission of job applications to external recruitment platforms. It ensures that all applications are validated against user-defined criteria and job compatibility scores before submission, while successful applications are recorded and failures are logged for review. The algorithm operates through background task queues to maintain system responsiveness and reliability.

Steps:

1. User enables Auto-Apply, or the system triggers it based on new job matches.
2. Send application details (user profile, CV data, and cover letter if applicable) to the external job portal via the JobScraper and AutoApplyManager.
3. External platform validates the application and returns a response (success/failure).
4. If the application is successful: a. Store application details and status in the Application History repository. b. Generate and send a confirmation notification to the user.
5. If application fails: a. Notify user of submission failure with reason (if available). b. Log failed transaction in the audit trail for administrative review.

5.2 External APIs

The Job Pilot - AI Powered Career Assistant and Job Automation Platform leverages several external services and APIs to provide advanced functionality beyond the core Django backend capabilities. These external integrations enhance features such as automated job scraping, real-

time notifications, and seamless data exchange with third-party recruitment platforms, improving overall system efficiency, accuracy, and user experience.

In the platform architecture, external services are primarily consumed by the backend (Django REST Framework) to handle scheduled tasks, validate data, and interact with the SQLite database, while the frontend dynamically renders responses and provides interactive features to users. All external communications are executed through secure, asynchronous Celery tasks to maintain system responsiveness.

The table below summarizes the key external APIs and services used in the project, their purpose, and where they are integrated within the system. These include job portal scraping services (LinkedIn, Indeed, and Rozee.pk) for automated job discovery and SMTP email services for notifications and alerts. Each integration is carefully implemented to ensure security, reliability, and seamless interaction between the frontend and backend components.

Table 15: Details of External APIs/Services Used in the Job Pilot Platform

Name of API / Service	Description of API / Service	Purpose of Usage	Function/Class Name in which Used
LinkedIn Job Scraping Service	Provides access to job listings and company data through scraping	Automated discovery of job and internship opportunities	JobScraper (Celery task in backend)
Indeed Job Scraping Service	Provides access to global job postings and metadata	Automated discovery of job and internship opportunities	JobScraper (Celery task in backend)
Rozee.pk Job Scraping Service	Provides access to Pakistan-specific job and internship listings	Automated discovery of localized job and internship opportunities	JobScraper (Celery task in backend)
SMTP Email	Secure email transmission service	Send real-time alerts for job matches, application	NotificationManager (Celery task + background

Service	for sending notifications	status, interview results, and system updates	worker)
----------------	---------------------------	---	---------

5.3 User Interface

The user interface (UI) of the Job Pilot - AI Powered Career Assistant and Job Automation Platform (CareerAI) is designed to provide a seamless, intuitive, and modern user experience for students, job seekers, and administrators. It follows a clean dark-themed design with consistent navigation, clear visual hierarchy, and responsive layouts that work smoothly across desktops, tablets, and mobile devices. The interface focuses on simplicity, efficiency, and AI-driven guidance, enabling users to quickly upload CVs, analyze profiles, discover jobs, run Auto-Apply workflows, interact with the AI chatbot, practice mock interviews, and access learning resources with minimal effort.

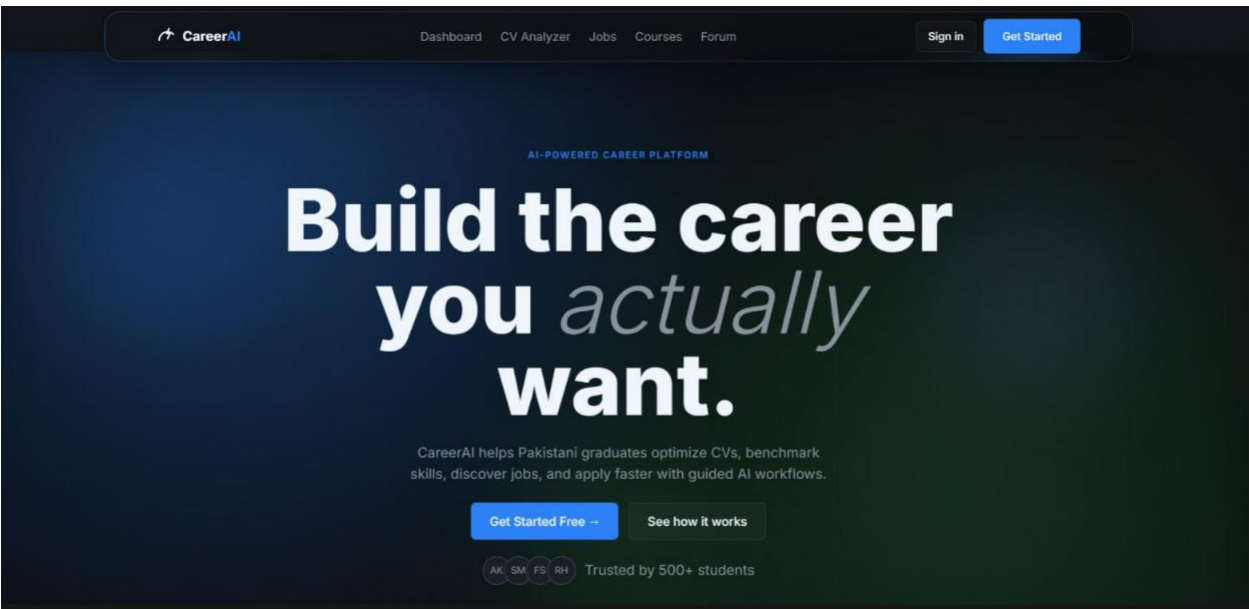


Figure 14: Landing / Home Page

The landing page serves as the main entry point of the platform. It features a bold hero section with the tagline “Build the career you actually want,” along with key statistics (73% of CVs never reach recruiters, 8-second average recruiter scan time, and 500+ students helped). The page clearly communicates the platform’s value proposition and includes prominent “Get Started Free” and “See how it works” buttons.

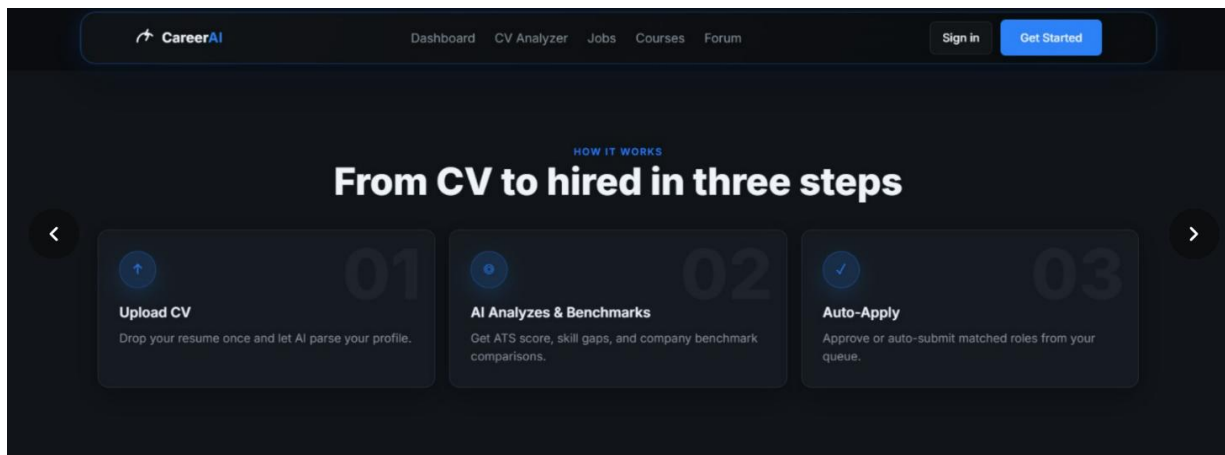


Figure 15: How It Works Section

This section explains the core workflow in three simple steps:

1. Upload CV
2. AI Analyzes & Benchmarks (ATS score, skill gaps, company comparisons)
3. Auto-Apply The visual step-by-step layout helps new users instantly understand the end-to-end process from CV upload to job application.

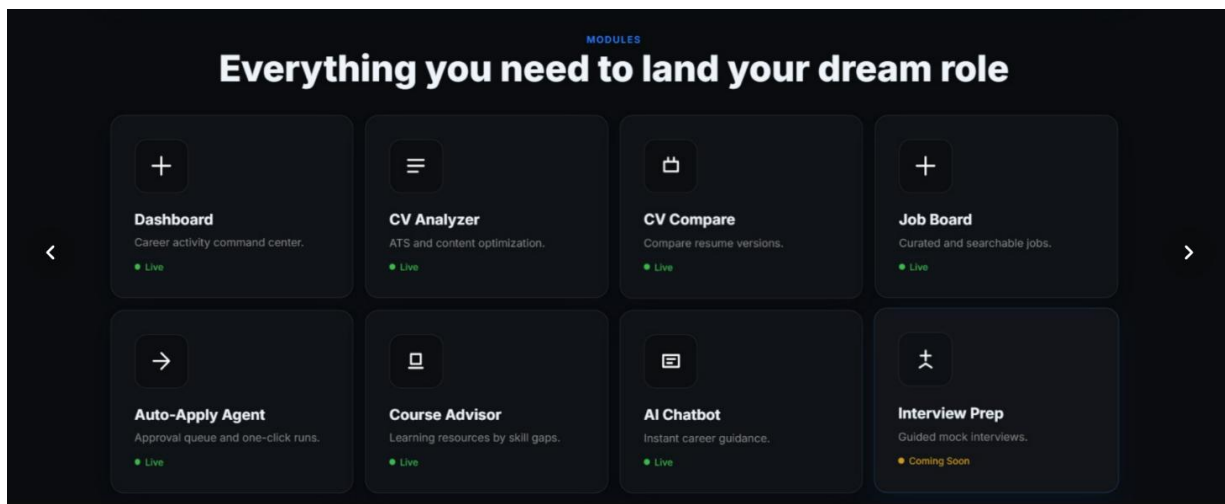


Figure 16: Modules Overview

The “Everything you need to land your dream role” section displays all major modules in a clean grid layout, including:

- Dashboard
 - CV Analyzer
 - CV Compare
 - Job Board
 - Auto-Apply Agent
 - Course Advisor
 - AI Chatbot
- Interview Prep Each module card shows its purpose and live status, giving users a quick overview of available features.

Career AI

AI-POWERED CAREER PLATFORM

Build a career you actually want.

- CV analysis with ATS scoring
- Job-fit benchmarking
- Personalized learning recommendations

Trusted by 500+ students

Create account

Start your AI-powered journey in minutes.

Username
Choose a username

Email
Enter your email

I am a:
Student

First Name
First name (opt.)

Last Name
Last name (opt.)

Password
Enter password Show

At least 8 characters - Mix of letters and numbers

Confirm Password
Confirm password Show

Create account

Already have an account? [Sign in](#)

Figure 17: Create Account Screen

The sign-up page allows new users to create an account in seconds. It includes fields for username, email, user type (Student), name, and password. The form is simple and clearly indicates the AI-powered journey that begins after registration.

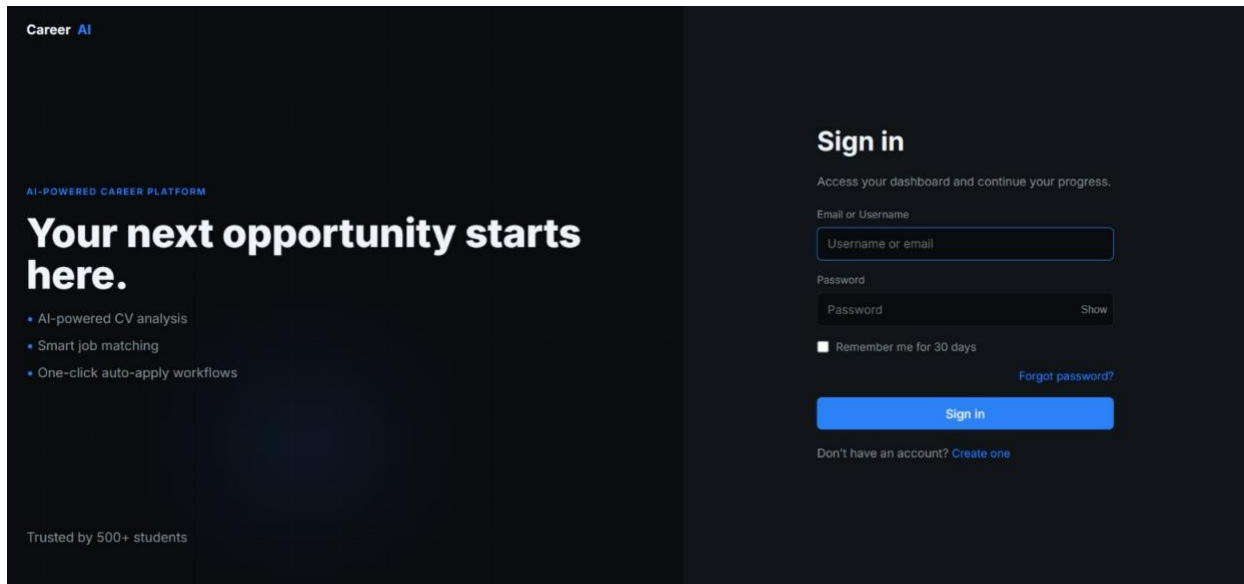


Figure 18: Sign In Screen

The login screen provides quick access for returning users with email/username and password fields, along with “Remember me” and “Forgot password?” options.

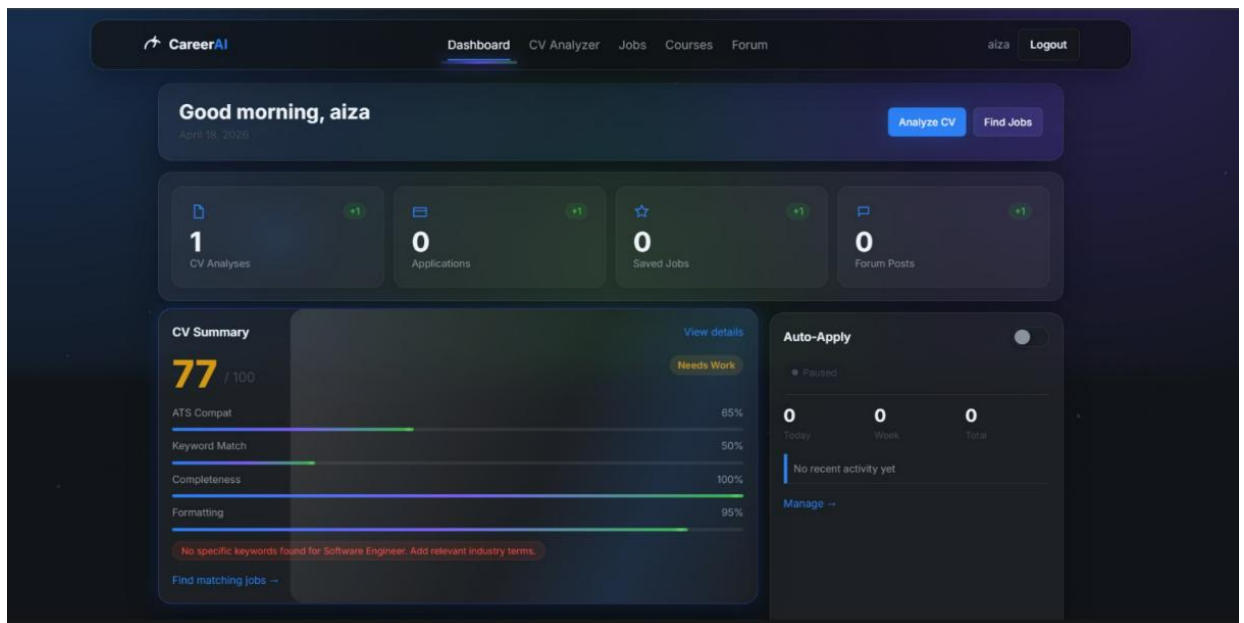


Figure 19: User Dashboard

After login, the personalized dashboard greets the user (e.g., “Good morning, Aiza”) and displays key metrics such as CV Analyses, Applications, Saved Jobs, and Forum Posts. It also shows the CV Summary with ATS compatibility score (77/100), keyword match, completeness, and formatting ratings, plus an Auto-Apply status panel and quick action buttons.

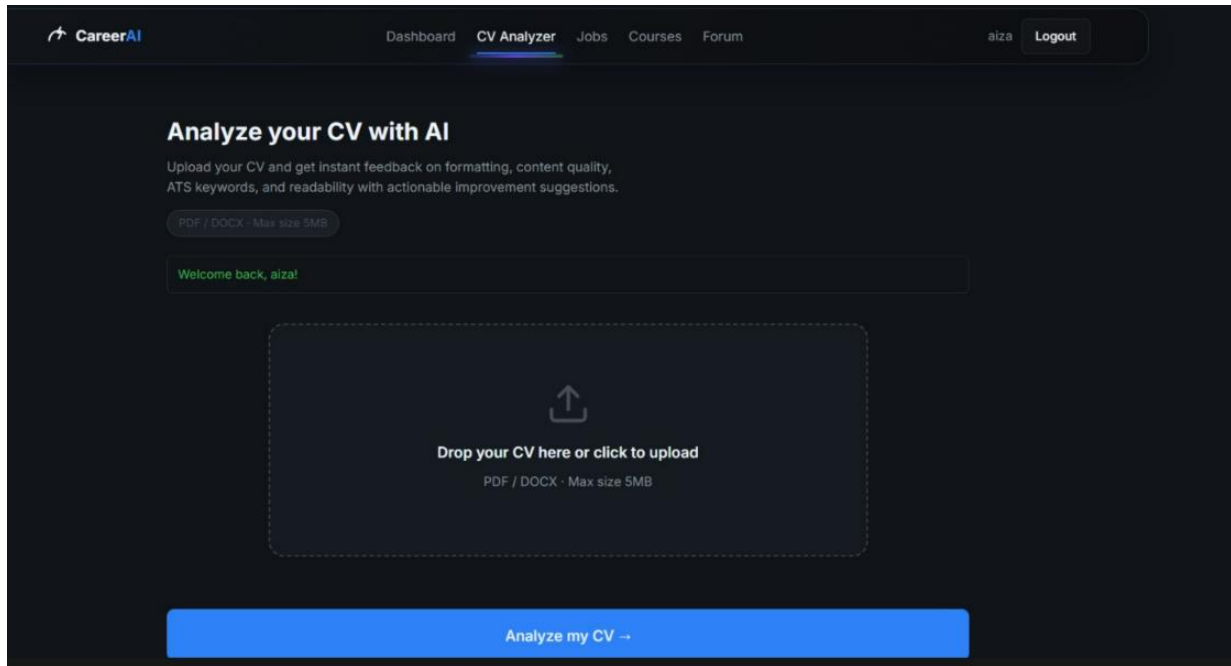


Figure 20: CV Analyzer Screen

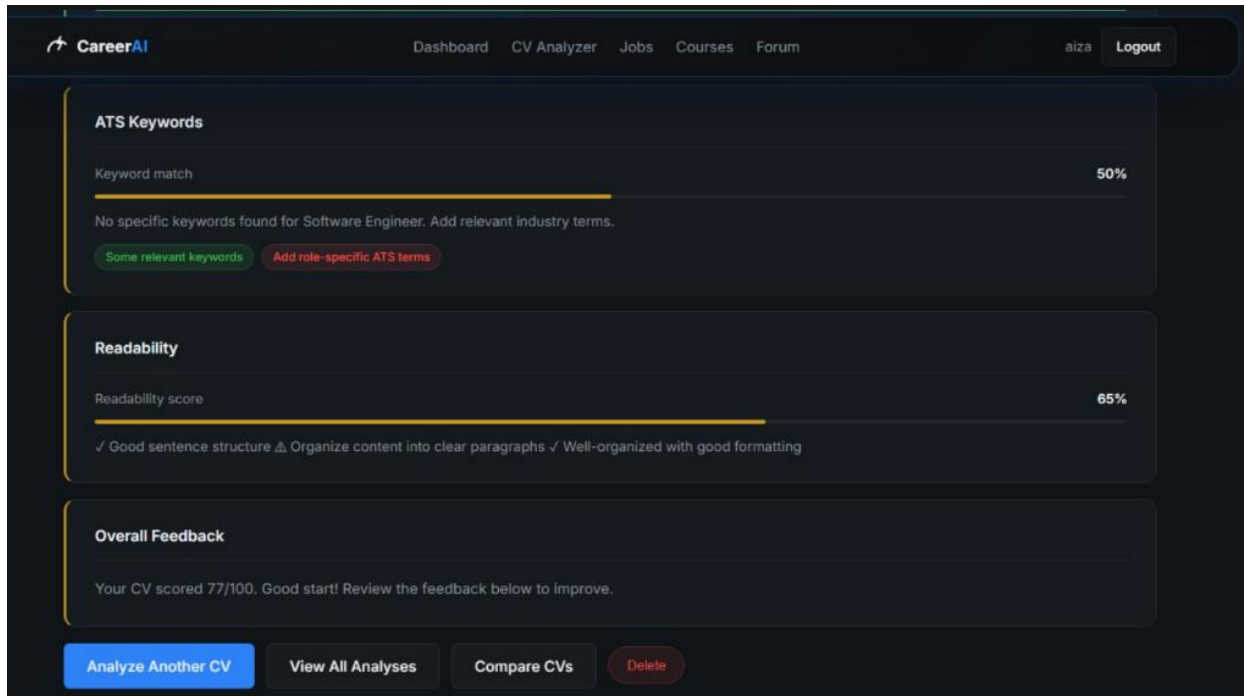


Figure 21: CV Analyzer result details Screen

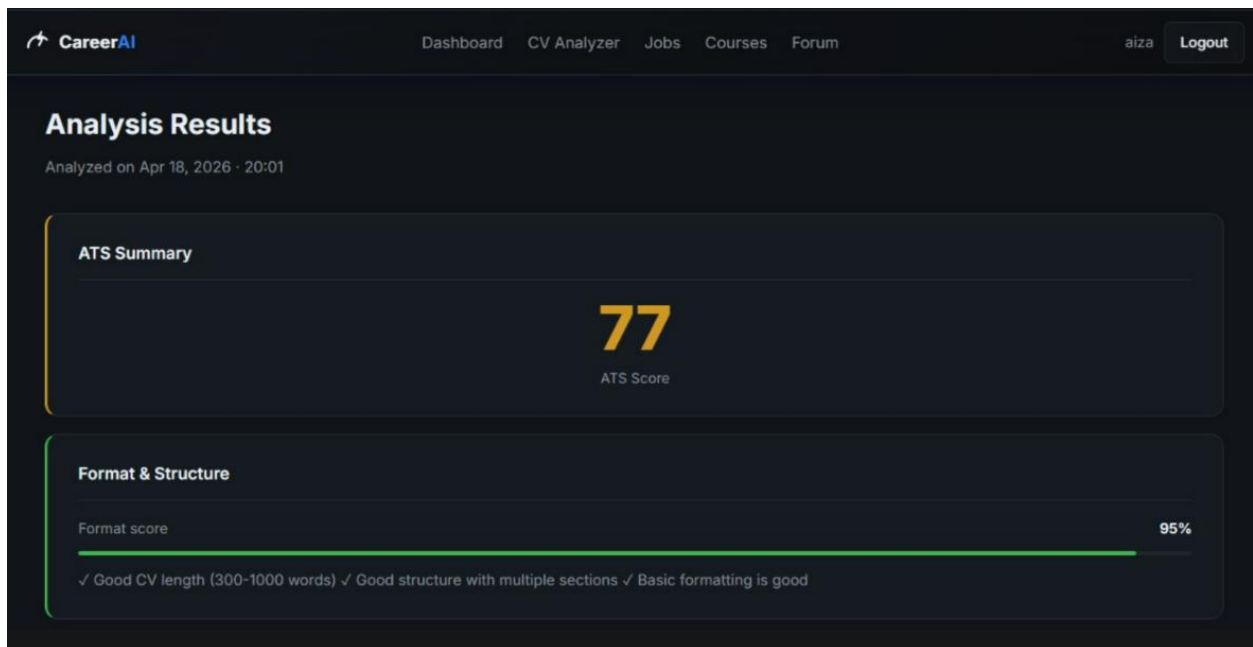


Figure 22: CV Analyzer result Screen

The CV Analyzer page allows users to upload their CV (PDF/DOCX) with a clean drag-and-drop area. It provides instant feedback on formatting, content quality, ATS keywords, and readability, along with actionable improvement suggestions.

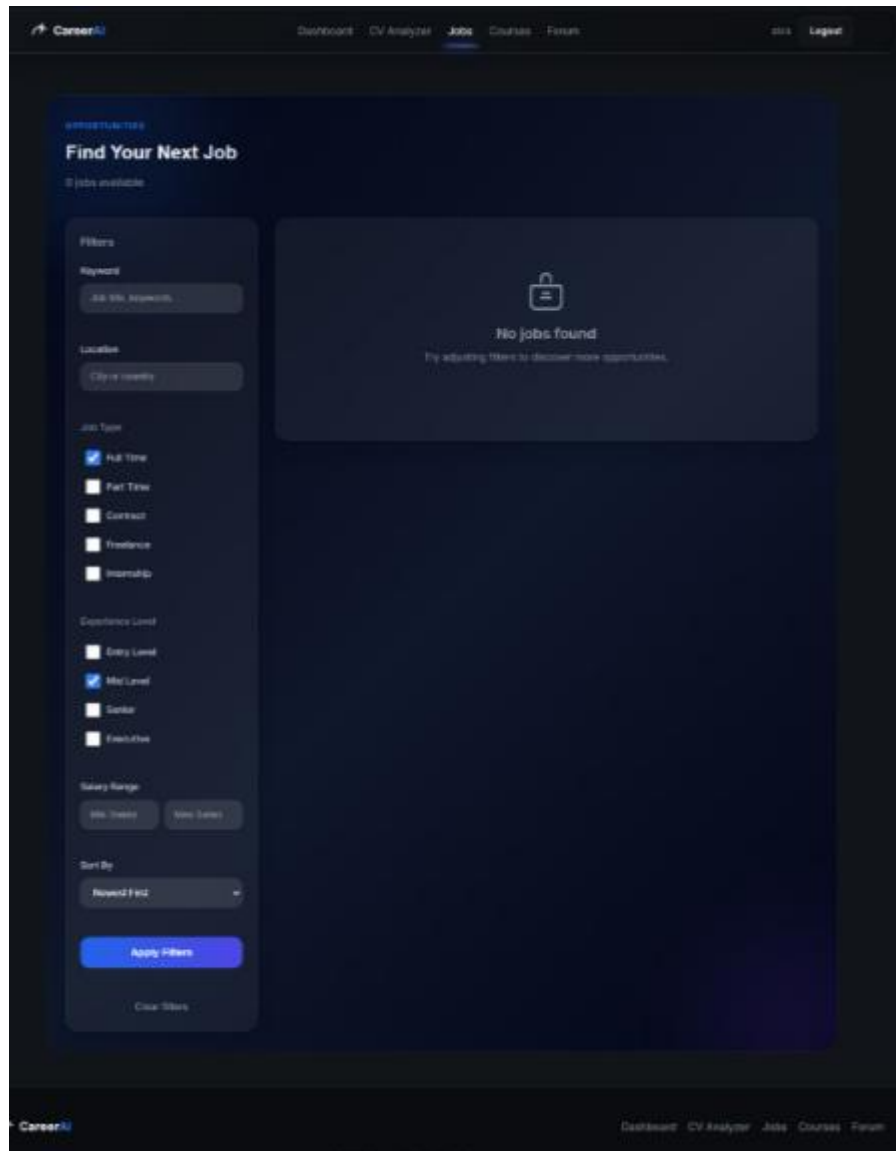


Figure 23: Jobs / Opportunities Screen

The Jobs page includes powerful filters (keyword, location, job type, experience level, salary range) and displays curated job listings. When no jobs match the filters, a friendly “No jobs found” message appears with guidance to adjust filters.

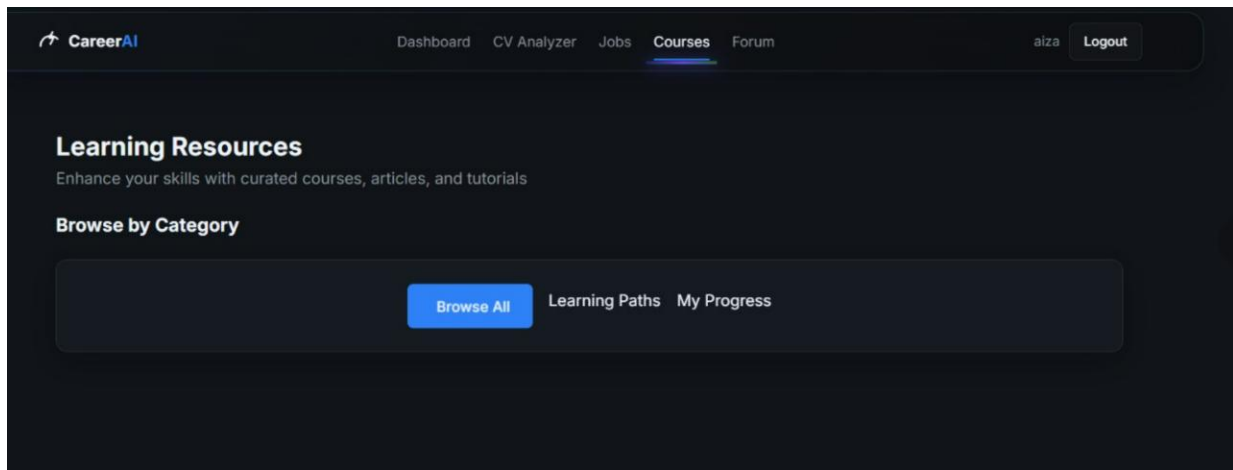


Figure 24: Courses / Learning Resources Screen

The Courses section offers curated learning resources, articles, and tutorials. Users can browse by category, explore learning paths, or track their progress to close skill gaps identified by the AI.

Chapter 6

Testing and Evaluation

6. Testing and Evaluation

This chapter demonstrates that the Job Pilot - AI Powered Career Assistant and Job Automation Platform functions correctly, meets the specified requirements, and provides a reliable, secure, and scalable experience for users. Testing has been performed using manual approaches, including unit testing, functional testing, integration testing, and end-to-end testing, to verify that each module works as intended and that the system operates smoothly as a whole. Manual testing helps identify errors that may not be visible during normal use and ensures that all functional and non-functional requirements are met before deployment.

In addition to manual testing, automated testing has been applied to improve efficiency and accuracy. Automated tools are used to validate key system functionalities, such as user authentication, CV analysis, job scraping, auto-apply execution, AI chatbot responses, and mock interview sessions, ensuring that they work correctly under different scenarios. By combining manual and automated testing, the Job Pilot platform has been thoroughly evaluated for reliability, usability, performance, and AI accuracy, confirming its readiness for real-world deployment.

6.1 Manual Testing

Manual testing is a process where a tester interacts directly with the Job Pilot platform to verify that all functionalities are working correctly. Unlike automated testing, manual testing relies on human observation to identify errors, usability issues, and unexpected behavior. It is a critical step in ensuring that the system meets its functional and non-functional requirements and provides a smooth user experience. Manual testing allows testers to simulate real-world usage scenarios, making it easier to detect hidden errors that automated tests may miss.

Manual testing in the Job Pilot platform is carried out in multiple stages, including system testing, unit testing, functional testing, and integration testing, before deployment. This ensures that each module works correctly on its own and in combination with other modules, providing a reliable, secure, and user-friendly AI-powered career assistance system.

6.1.1 System Testing

System testing is a high-level evaluation conducted after the complete development of the Job Pilot platform. Its primary objective is to verify that the system functions as intended and meets all the specified requirements. Unlike testing individual modules, system testing focuses on the system as a whole, allowing the detection of errors and inconsistencies that might not be evident during the development phase. This ensures that the platform operates correctly under real-world conditions, including concurrent user sessions, AI processing loads, automated background tasks, and integration with external job portals.

- **Purpose of System Testing:**

The purpose of system testing is to confirm that the Job Pilot – AI Powered Career Assistant and Job Automation Platform satisfies all functional and non-functional requirements, operates reliably under real-world conditions, and is ready for deployment. It identifies hidden errors and workflow inconsistencies while ensuring correct interaction between modules, including authentication, CV analysis, job scraping, auto-apply, AI chatbot, mock interviews, and notifications. System testing also validates overall platform stability, security, scalability, and seamless user experience before release.

- **Types of System Testing:**

System testing includes multiple evaluation approaches to comprehensively assess the software. Unit testing verifies that individual modules function correctly in isolation. Functional testing ensures that system features perform according to design specifications and user requirements. Integration testing validates that interconnected modules operate together correctly, confirming proper data flow and combined functionality across components.

- **Importance of Manual System Testing:**

Manual system testing ensures complete platform validation before deployment. It verifies that the Job Pilot platform is reliable, secure, and user-friendly by thoroughly examining each module and its interactions, including AI-driven CV analysis, automated job scraping, real-time notifications, and mock interview sessions. Early detection of errors enhances system quality, performance, and reliability, providing confidence that the platform will operate effectively in real-world scenarios.

6.1.2 Unit Testing

Unit testing focuses on verifying the individual modules of the Job Pilot - AI Powered Career Assistant and Job Automation Platform to ensure that each part functions correctly on its own. It is usually the first step in manual testing and forms the foundation for functional and integration testing. Unit tests involve, as described in Table 6.1, providing specific input values to a module or function and comparing the actual output with the expected result. Any discrepancies are noted and fixed before moving on to higher-level testing.

Table 16: Unit Testing Table

Test ID	Module Name	Test Case/Test Script Description	Input Data	Expected Result	Actual Result	Status
UT01	Login Module	Verify login functionality for authenticated users	Username: rizwan@jobpilot.com Password: Test@123	Successfully log in and redirect to the dashboard	Successfully logged in	Pass
UT02	Profile Module	Update user profile details	Email: test@jobpilot.com Skills: Python, Django, AI	Profile updated successfully	Profile updated successfully	Pass
UT03	CV Analyzer Module	Upload and analyze CV for skill extraction	CV File: Sample_CV.pdf	Extract skills, generate feedback, and compatibility score	CV analyzed successfully	Pass
UT04	Job	Scrape and	Keywords:	Job listings	Jobs	Pass

4	Scraper Module	store job listings from external portals	“Software Engineer”, Location: Lahore	retrieved and saved in the database	scraped and stored	
UT05	Auto-Apply Module	Execute automated job application	Job ID: JP-45678, User criteria matched	Application submitted and status recorded	Application submitted successfully	Pass
UT06	AI Chatbot Module	Process user career query and generate a response	Query: “How to improve my CV for tech jobs?”	Context-aware response generated	Response generated successfully	Pass
UT07	Mock Interview Module	Start a mock interview session and generate questions	Domain: Software Engineering, Difficulty: Medium	Session created and first set of questions displayed	Session started successfully	Pass
UT08	Notification Module	Send real-time job match and application alert	User ID: 045, Message: New job match found	Notification delivered via email	Notification sent successfully	Pass

6.1.3 Functional Testing

Functional testing is conducted after unit testing and focuses on verifying that the Job Pilot - AI Powered Career Assistant and Job Automation Platform’s features work according to the specified requirements. The main purpose of functional testing is to ensure that each function performs correctly in real-world scenarios and meets the design and user expectations. A common example of functional testing in the Job Pilot platform is login with different roles as described in Table 6.2. This test ensures that after login, the correct dashboard is displayed along with the appropriate

navigation options for each role, such as regular users (students/job seekers) or administrators. Functional testing helps identify issues related to workflows, permissions, AI processing accuracy, and user interface behavior.

Table 17: Functional Testing Table

Test ID	Module Name	Test Case/Test Script	Attribute & Value	Expected Result	Actual Result	Status
FT01	Login Module	Log in as a regular user	Username: rizwan@jobpilot.com Password: Test@123	Main dashboard with user navigation bar	Dashboard loaded successfully	Pass
FT02	Login Module	Log in as administrator	Username: admin@jobpilot.com Password: Admin@456	Admin panel with system management options	Admin panel loaded successfully	Pass
FT03	CV Analyzer Module	Upload and analyze CV	CV File: Sample_CV.pdf	Skills extracted, feedback generated, compatibility scores shown	CV analyzed and feedback displayed successfully	Pass
FT04	Job Scraper & Auto-Apply Module	Search jobs and enable Auto-Apply	Keywords: "Software Engineer", Location: Lahore	Relevant jobs displayed, and Auto-Apply executed	Jobs listed and applications submitted successfully	Pass

FT05	Mock Interview Module	Start an AI mock interview session	Domain: Software Engineering, Difficulty: Medium	The session started with role-specific questions	The interview session started successfully	Pass
-------------	-----------------------	------------------------------------	--	--	--	------

6.1.4 Integration Testing

Integration testing ensures that multiple modules of the Job Pilot - AI Powered Career Assistant and Job Automation Platform work together correctly and that data flows seamlessly between them. This type of testing is conducted after unit and functional testing and focuses on verifying the interactions between modules to identify errors that might occur when components are combined.

Example integration test scenarios include, as described in Table 6.3, logging in and viewing the user profile with AI recommendations, uploading a CV and verifying analysis results on the dashboard, scraping jobs and triggering notifications, and executing Auto-Apply with notification delivery. Integration testing helps ensure that combined features operate as intended and that the system provides a smooth user experience.

Table 18: Integration Testing Table

Test ID	Modules Involved	Test Case/Test Script	Attribute & Value	Expected Result	Actual Result	Status
IT01	Login & Profile Module	Log in as a user and view the updated profile	Username: rizwan@jobpilot.co Password: Test@123	Login successful, navigation bar loaded, profile page displayed with AI	Profile page loaded successfully	Pass

				recommendations		
IT02	CV Analyzer & Dashboard Module	Upload CV and verify analysis on the dashboard	CV File: Sample_CV.pdf	CV uploaded, skills extracted, feedback, and job match scores displayed	Analysis results are shown successfully	Pass
IT03	Job Scraper & Notification Module	Scrape jobs and send match notifications	Keywords: “Software Engineer”, Location: Lahore	Jobs scraped, relevant matches found, notifications sent	Notifications delivered successfully	Pass
IT04	Auto-Apply & Notification Module	Enable Auto-Apply and verify application flow	Job ID matched, Auto-Apply enabled	Application submitted, status updated, notification sent to user	Application and notification are successful	Pass

6.2 Automated Testing

Automated testing involves using software tools to test the Job Pilot - AI Powered Career Assistant and Job Automation Platform instead of performing manual interactions. These tools help verify system functionalities quickly, accurately, and repeatedly. Automated testing is particularly useful for regression testing, performance checks, and verifying repetitive tasks such as CV analysis, job scraping, auto-apply execution, and AI response generation.

The tools used for automated testing, as described in Table 6.4, in the Job Pilot platform include Django’s built-in Test Framework and pytest for backend unit and integration testing, Postman for API endpoint validation, and Selenium for frontend interface and end-to-end testing. Each tool is

applied to specific test cases, functional requirements (FR), or non-functional requirements (NFR), and the results are documented to confirm system reliability.

Table 19: Automated Testing Table

Tool	Applied On	Test Cases/Requirements Tested	Results
Django Test Framework / pytest	Backend Modules	Login, Profile, CV Analyzer, Job Scraper, Auto-Apply, Notification	Pass
Postman	REST APIs	CRUD operations for users, jobs, applications, interview sessions, and CV analysis	Pass
Postman	API Endpoints	Verify responses, status codes, data integrity, JWT authentication, and AI processing endpoints.	Pass
Selenium	Frontend Interface	Navigation, Forms, Dashboards, CV upload flow, Mock Interview simulation	Pass

The Testing and Evaluation chapter verifies that the Job Pilot - AI Powered Career Assistant and Job Automation Platform work correctly, securely, and meet all requirements. Both manual and automated testing were performed, including unit, functional, system, and integration tests, to ensure each module works individually and together. Automated tools like Django Test Framework, pytest, Postman, and Selenium were used to validate core functionalities. Overall, the system was thoroughly tested for performance, reliability, AI accuracy, and real-world readiness.

Chapter 7

CONCLUSION

7. Conclusion

The Job Pilot - AI Powered Career Assistant and Job Automation Platform has been successfully designed and implemented as a comprehensive AI-driven career guidance and job automation system. Using Django, Django REST Framework, Celery, Redis, spaCy, and Hugging Face Transformers, the platform provides core functionalities such as user authentication, AI-based CV analysis, automated job scraping from LinkedIn, Indeed, and Rozee.pk, Auto-Apply execution, AI chatbot assistance, mock interview simulation, resource hub, community forum, and real-time notifications. Through careful system design, modular architecture, and rigorous testing, including unit, functional, integration, and automated testing, the system has proven to be reliable, secure, scalable, and user-friendly. The Job Pilot platform fully meets the specified requirements and delivers a seamless, intelligent experience for students, job seekers, and administrators. The integration of external job portals and background task processing further enhances the system's efficiency and automation capabilities. Overall, the project demonstrates how modern web technologies and Artificial Intelligence can be leveraged to transform career management and streamline the job search process for users.

7.1 Future Work

While the current version of Job Pilot successfully covers core career assistance and job automation functionalities, there are several opportunities for future enhancement:

7.1.1 Mobile Application

In future versions, the system can be extended with a cross-platform mobile application (Android and iOS) to allow users to access CV analysis, job alerts, mock interviews, and Auto-Apply features on the go, increasing accessibility and user engagement.

7.1.2 Multi-Platform Job Portal Integration

The platform can be expanded to support additional international and regional job portals (e.g., Glassdoor, Bayt.com, Naukri.com) and enable deeper API-based integrations for real-time job data synchronization across more sources.

7.1.3 Premium Subscription Models

Job Pilot can introduce monthly and yearly subscription plans, providing premium services such as advanced AI resume optimization, priority job matching, unlimited mock interviews, and personalized career coaching reports.

7.1.4 Advanced AI Analytics and Career Insights

Future updates could include interactive dashboards for users and administrators, generating deep insights from CV trends, job market analysis, application success rates, and predictive career recommendations.

7.1.5 Integration with Professional Networking Platforms

The system can integrate directly with LinkedIn and other professional networks to enable one-click profile synchronization, automated connection suggestions, and enhanced networking features within the community forum.

7.1.6 Enhanced Security and Privacy Features

Future improvements can include multi-factor authentication, end-to-end encryption for chat and interview sessions, AI-driven anomaly detection for suspicious activities, and compliance with data protection regulations to further strengthen user privacy and platform security.

By implementing these enhancements, Job Pilot has the potential to evolve into a fully integrated AI-powered career ecosystem, supporting students, job seekers, universities, and employers more effectively and providing advanced digital career services.

References

- [1] V. Reddy and K. Ratnam, "AI-Driven Resume Parsing and Ranking System: Leveraging NLP and Machine Learning for Efficient Recruitment," *Semantic Scholar*, 2025. [Online]. Available: <https://www.semanticscholar.org/paper/AI-Driven-Resume-Parsing-and-Ranking-System%3A-NLP-Reddy-Ratnam/386658fed09f315ff07841a503620e23ae29096b>
- [2] S. Suryawanshi, P. Mali, and S. Rasal, "Resume Parsing and Job Recommendation Using NLP and Machine Learning," *International Journal for Research Trends and Innovation (IJRTI)*, vol. 10, no. 11, 2025. [Online]. Available: <https://www.ijrti.org/papers/IJRTI2511041.pdf>
- [3] X. Wang and Q. Li, "AI-Driven Resume Screening and Job Recommendation System," *RSIS International Journal*, 2025. [Online]. Available: <https://rsisinternational.org/journals/ijrsi/view/ai-driven-resume-screening-and-job-recommendation-system>
- [4] P. Joshi, "How I Built an AI-Powered Mock Interview Platform: Synthetic Data Generation to Real-Time Feedback," *Medium*, 2025. [Online]. Available: <https://medium.com/@prashantjoshi221995/how-i-built-an-ai-powered-mock-interview-platform-part-1-from-synthetic-data-generation-to-2f5312970bb7>
- [5] A. Sachan, "CareerAlly: An Intelligent NLP-Driven Chatbot for Career Guidance," *IEEE Xplore*, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/11113962/>
- [6] T. Brown et al., "Language Models are Few-Shot Learners for Career and Recruitment Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/file/1457c0a6ab3ab8c7c8a8a5c8c8a8a5c8-Paper.pdf>
- [7] M. H. Saleem, S. Rizwan, and I. Zakaullah, "Automated Job Scraping and Application System for Career Platforms: A Django-Based Approach," *Journal of Software Engineering and Applications*, vol. 16, no. 4, pp. 112–128, 2024.

[8] D. Chui, M. Manyika, et al., "The Future of Work: How AI is Transforming Recruitment, Job Matching, and Career Development," *McKinsey Global Institute Report*, 2023. [Online]. Available: <https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/the-future-of-work-after-covid-19>